

附录 A 支撑材料文件列表

支撑材料以 ZIP 压缩包单独提交。为便于评阅和复核,表 1 仅列出文件类别、核心入口及用途;完整目录结构、运行命令、依赖版本和随机参数见压缩包根目录的 README.md。赛题提供的原始附件不重复列入自主数据资料。

表 1: 支撑材料文件列表

序号	文件或目录	内容及用途
1	README.md、各问题 README.md	总体目录、运行环境、运行顺序、输入输出与结果复现说明。
2	requirements.txt、 environment.yml、config.json	Python 依赖、Conda 环境及问题一搜索参数。
3	question1/program/run.py	问题一统一运行入口: 机队规模搜索、路线优化、结果输出与复核。
4	question1/program/src/uav_q1/	问题一启发式求解、精确模型、独立校验及 Excel 输出模块;完整代码见附录 B。
5	question1/results/	问题一正式工作簿、结构化路线和搜索记录。
6	question2/program/q2_solver.py	问题二负载均衡求解与独立校验程序;完整代码见附录 C。
7	question2/results/	问题二工作簿、结构化方案、汇总指标和逐架路线。
8	question3/program/q3_solver.py	问题三算法 A: 直飞、必要等待与路线优化。
9	question3/program/q3_solver_enhanced.py	问题三算法 B: 时变可见图、空间绕飞与两阶段优化。
10	question3/program/q3_fast_validator.py、 q3_enhanced_validator.py	算法 A 快速校验与算法 B 逐事件独立复核;完整代码见附录 D。
11	question3/results/	算法 A 对照结果、算法 B 正式结果及机队规模敏感性结果。
12	generated_figures/、各问题辅助绘图与工作簿构建程序	论文图件、结果整理和格式转换文件,仅作为电子支撑材料提交。
13	AI .pdf	人工智能工具名称、用途、人工修改和核验情况。

结果口径:问题三正式方案采用算法 B,固定问题一最终采用的机队规模 $N_0 = (4, 2, 5, 4)$;算法 A 仅作基准对照, $N_0 + 1$ 、 $N_0 + 2$ 仅用于敏感性分析。全部正式结果均由独立程序复核。验证通过仅说明方案满足本文实现的约束,不代表已经证明全局最优。

附录 B 问题一源程序

以下五个文件共同构成问题一完整程序, 运行入口为 `run.py`。测试脚本、环境安装脚本、说明文档和预计算结果仅放入电子支撑材料。

问题一统一运行入口

路径: `submission_package/question1/program/run.py`

```

1
2 from __future__ import annotations
3
4 import argparse
5 import json
6 import math
7 import os
8 import sys
9 import time
10 from pathlib import Path
11
12 ROOT = Path(__file__).resolve().parent
13 sys.path.insert(0, str(ROOT / "src"))
14
15 # 多进程随机起点优先, 避免每个工作进程再启动一组 BLAS/OpenMP 线程。
16 os.environ.setdefault("OMP_NUM_THREADS", "1")
17 os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
18 os.environ.setdefault("MKL_NUM_THREADS", "1")
19
20 import numpy as np
21 import pandas as pd
22 from scipy.optimize import linear_sum_assignment
23 from scipy.sparse.csgraph import minimum_spanning_tree
24
25 from uav_q1.exact_milp import result_as_dict, solve_fixed_n_exact
26 from uav_q1.excel_output import export_workbook
27 from uav_q1.solver_engine import REQ, SERVICE_H, SPEED, UNIT_KM,
    ↳ dist, rtime, solve
28 from uav_q1.validation import validate_all
29
30
31 def load_config() -> dict:
32     return json.loads((ROOT /
    ↳ "config.json").read_text(encoding="utf-8"))
33
34
35 def pack_case(case_name: str, df: pd.DataFrame, routes:
    ↳ list[list[int]]) -> dict:
36     points = df[["X_Coordinate", "Y_Coordinate"]].to_numpy(float)
37     packed = []
38     for uid, route in enumerate(routes, 1):
39         point_ids = [int(df.iloc[i].Point_ID) for i in route]
40         packed.append({
41             "uav": uid,
42             "sequence": [0, *point_ids, 0],
43             "distance_km": dist(route, points) * UNIT_KM,
44             "service_count": len(route),
45             "time_h": rtime(route, points),
46         })
47     packed.sort(key=lambda x: x["time_h"], reverse=True)
48     for uid, route in enumerate(packed, 1):
49         route["uav"] = uid
50     return {
51         "N": len(packed),
52         "Tmax": max(x["time_h"] for x in packed),
53         "Tmin": min(x["time_h"] for x in packed),
54         "routes": packed,
55     }
56
57
58 def mst_lower_bound(df: pd.DataFrame, limit_h: float) -> int:
59     points = df[["X_Coordinate", "Y_Coordinate"]].to_numpy(float)
60     visits = df.Inspection_Level.map(REQ).to_numpy(int)
61     augmented = np.vstack([[0.0, 0.0], points])
62     matrix = np.linalg.norm(augmented[:, None, :] - augmented[None,
    ↳ :, :], axis=2) * UNIT_KM
63     mst_km = float(minimum_spanning_tree(matrix).sum())
64     total_lower_h = visits.sum() * SERVICE_H + mst_km / SPEED
65     return max(1, math.ceil(total_lower_h / limit_h - 1e-12))
66
67
68 def _cycle_cover_lower_bound_km(df: pd.DataFrame, n_uav: int) ->
    ↳ float:
69     """ 任务副本循环覆盖松弛; 同物理点副本间以及基地副本间禁止直连。"""
70     physical = np.repeat(
71         np.arange(len(df), dtype=int),
72         df.Inspection_Level.map(REQ).to_numpy(int),
73     )
74     points = df[["X_Coordinate", "Y_Coordinate"]].to_numpy(float)
75     task_xy = points[physical]
76     m = len(physical)
77     size = m + int(n_uav)
78     large = 1.0e12
79     cost = np.full((size, size), large, dtype=float)
80     task_cost = np.linalg.norm(task_xy[:, None, :] - task_xy[None, :,
    ↳ :], axis=2) * UNIT_KM
81     task_cost[physical[:, None] == physical[None, :]] = large
82     cost[:, m:] = task_cost
83     depot = np.linalg.norm(task_xy, axis=1) * UNIT_KM
84     cost[:, m:] = depot[:, None]
85     cost[m:, m] = depot[None, :]
86     row, col = linear_sum_assignment(cost)
87     chosen = cost[row, col]
88     return math.inf if np.any(chosen >= large / 2) else
    ↳ float(chosen.sum())
89
90
91 def strong_lower_bound(df: pd.DataFrame, limit_h: float) -> dict:
92     points = df[["X_Coordinate", "Y_Coordinate"]].to_numpy(float)
93     demand = df.Inspection_Level.map(REQ).to_numpy(int)
94     augmented = np.vstack([[0.0, 0.0], points])
95     matrix_units = np.linalg.norm(augmented[:, None, :] -
    ↳ augmented[None, :, :], axis=2)
96     mst_km = float(minimum_spanning_tree(matrix_units).sum()) *
    ↳ UNIT_KM
97     repeated_degree_units = 0.0
98     for idx, count in enumerate(demand, start=1):
99         incident = np.delete(matrix_units[idx], idx)
100         repeated_degree_units += int(count) *
    ↳ float(np.partition(incident, 1)[:2].sum())
101     repeated_degree_km = repeated_degree_units * UNIT_KM / 2.0
102     service_h = float(demand.sum()) * SERVICE_H
103     farthest_h = float(np.max(2.0 * matrix_units[0, 1:] * UNIT_KM /
    ↳ SPEED + SERVICE_H))
104     n = max(1, math.ceil(service_h / limit_h - 1e-12))
105     while True:
106         cycle_km = _cycle_cover_lower_bound_km(df, n)
107         routing_km = max(mst_km, repeated_degree_km, cycle_km)
108         aggregate_h = service_h + routing_km / SPEED
109         makespan_lb = max(aggregate_h / n, farthest_h)
110         if makespan_lb <= limit_h + 1e-12:
111             break
112         n += 1
113     return {
114         "N": int(n),
115         "service_hours": service_h,
116         "mst_km": mst_km,
117         "repeated_degree_km": repeated_degree_km,
118         "cycle_cover_km": cycle_km,
119         "aggregate_work_hours": aggregate_h,
120         "makespan_lower_bound_at_N": makespan_lb,
121         "farthest_roundtrip_lower_bound_hours": farthest_h,
122     }
123
124

```

```

125 def routes_from_precomputed(df: pd.DataFrame, case: dict) -> 188
    ↪ list[list[int]]:
126     id_to_index = {int(row.Point_ID): i for i, row in df.iterrows()}
127     routes = [] 189
128     for route in case["routes"]: 190
129         routes.append([id_to_index[int(point_id)] for point_id in 191
            ↪ route["sequence"] if int(point_id) != 0])
130     return routes 192
131 193
132 194
133 def solve_from_scratch(input_path: Path, quality: str, engine: str, 195
    ↪ config: dict) -> dict: 196
134     book = pd.ExcelFile(input_path) 197
135     results = {}
136     seeds = int(config["quality_seeds"][quality]) 198
137     for case_name in book.sheet_names: 199
138         df = pd.read_excel(book, sheet_name=case_name) 200
139         lower = mst_lower_bound(df, 201
            ↪ float(config["maximum_work_hours"])) 202
140         n = lower 203
141         limit_h = float(config["maximum_work_hours"]) 204
142         max_n = lower + int(config.get("maximum_n_increment", 6)) 205
143         history = []
144         all_smaller_proven_infeasible = True 206
145         print(f"[{case_name}] strict search starts at lower bound 207
            ↪ N={n}; quality={quality}; engine={engine}", flush=True) 208
146         while n <= max_n: 209
147             heuristic_routes = None 210
148             heuristic_obj = None 211
149             if engine in {"heuristic", "hybrid"}: 212
150                 (heuristic_obj, heuristic_routes), _, _ = 213
                    ↪ solve(case_name, df, n, seeds=seeds) 214
151                 print(f"[{case_name}] N={n}, heuristic best 215
                    ↪ Tmax={heuristic_obj:.6f} h", flush=True) 216
152                 record = { 217
153                     "N": n, 218
154                     "heuristic_best_Tmax": float(heuristic_obj), 219
155                     "heuristic_feasible": bool(heuristic_obj <= 220
                        ↪ limit_h + 1e-9), 221
156                 } 222
157                 history.append(record)
158                 if heuristic_obj <= limit_h + 1e-9: 223
159                     case = pack_case(case_name, df, heuristic_routes) 224
160                     case["theoretical_lower_bound_N"] = lower 225
161                     case["fleet_size_status"] = ( 226
                        ↪ "PROVEN_MINIMUM" if 227
                        ↪ all_smaller_proven_infeasible else 228
                        ↪ "FEASIBLE_CANDIDATE" 229
                    ) 230
162                     case["makespan_status"] = 231
                        ↪ "BEST_FOUND_NOT_PROVEN_OPTIMAL" 232
163                     case["search_history"] = history 233
164                     results[case_name] = case 234
165                     break
166
167         if engine in {"exact", "hybrid"}: 235
168             exact = solve_fixed_n_exact( 236
169                 df, 237
170                 n, 238
171                 limit_h=limit_h, 239
172                 time_limit_s=float(config.get("exact_time_limit_ 240
                    ↪ seconds", 300)), 241
173                 mip_rel_gap=float(config.get("exact_mip_relative 242
                    ↪ _gap", 0.0)), 243
174             ) 244
175             exact_record = result_as_dict(exact) 245
176             exact_record["N"] = n 246
177             if history and history[-1].get("N") == n: 247
178                 history[-1]["exact"] = exact_record 248
179             else: 249
180                 history.append({"N": n, "exact": exact_record}) 250
181             print(f"[{case_name}] N={n}, exact 251
                ↪ status={exact.status}, 252
                ↪ objective={exact.objective_h}", flush=True) 253
182             if exact.status in {"OPTIMAL", "FEASIBLE"} and 254
                ↪ exact.routes is not None: 255
183                 case = pack_case(case_name, df, exact.routes) 256
184                 case["theoretical_lower_bound_N"] = lower 257
185                 case["fleet_size_status"] = ( 258
                    ↪ "PROVEN_MINIMUM" if 259
                    ↪ all_smaller_proven_infeasible else 260
                    ↪ "FEASIBLE_CANDIDATE" 261
                ) 262
263                 case["makespan_status"] = ( 263
                    ↪ "PROVEN_OPTIMAL" if exact.status == "OPTIMAL" 264
                    ↪ else "FEASIBLE_NOT_PROVEN_OPTIMAL" 265
                ) 266
267                 case["search_history"] = history
268                 results[case_name] = case
269                 break
270
271             if exact.status != "INFEASIBLE":
272                 # UNKNOWN 不能当作不可行;
273                 ↪ 继续向上寻找只会得到可行上界。
274                 all_smaller_proven_infeasible = False
275             else:
276                 # 启发式失败也不能当作不可行证明。
277                 all_smaller_proven_infeasible = False
278                 n += 1
279             else:
280                 raise RuntimeError(
281                     f"[{case_name}]: 在 N={lower}..{max_n}
282                     ↪ 内没有找到可行方案;"
283                     ↪ "请提高搜索质量、精确求解时限或 maximum_n_increment"
284                 )
285         return results
286
287 def run_ten_minute_plan(
288     input_path: Path,
289     config: dict,
290     total_seconds: float,
291     workers: int,
292     candidate_k: int,
293     resume_from: Path | None = None,
294 ) -> tuple[dict, dict]:
295     """ 在统一墙钟预算内验证热启动并行并改进四个固定车队规模。"""
296     started = time.monotonic()
297     deadline = started + float(total_seconds)
298     reserve = min(float(config.get("ten_minute_save_reserve_seconds",
299         ↪ 45)), total_seconds * 0.2)
300     precomputed = json.loads((ROOT / "data" /
301         ↪ "precomputed_solution.json").read_text(encoding="utf-8"))
302     validate_all(input_path, precomputed, config)
303     if resume_from is not None:
304         resumed = json.loads(resume_from.read_text(encoding="utf-8"))
305         validate_all(input_path, resumed, config)
306         precomputed = resumed
307         print(f"续搜方案独立校验通过: {resume_from}", flush=True)
308     else:
309         print("已有 4,2,5,4 方案独立校验通过; 开始强下界与并行改进。",
310             ↪ flush=True)
311
312     book = pd.ExcelFile(input_path)
313     weights = {k: float(v) for k, v in
314         ↪ config.get("ten_minute_case_weights", {}).items()}
315     process_order = [name for name in ["Case2", "Case4", "Case1",
316         ↪ "Case3"] if name in book.sheet_names]
317     improved: dict[str, dict] = {}
318     report = {
319         "mode": "ten-minute",
320         "requested_total_seconds": float(total_seconds),
321         "workers": int(workers),
322         "candidate_neighborhood_size": int(candidate_k),
323         "gpu_used": False,
324         "cases": {},
325     }
326
327     for position, case_name in enumerate(process_order):
328         df = pd.read_excel(book, sheet_name=case_name)
329         lower = strong_lower_bound(df,
330             ↪ float(config["maximum_work_hours"]))
331         initial = precomputed[case_name]
332         n_uav = int(initial["N"])
333         if n_uav < int(lower["N"]):
334             raise AssertionError(f"[{case_name}]: 已有 N={n_uav}
335                 ↪ 小于严格下界 {lower['N']}")
336         warm_routes = routes_from_precomputed(df, initial)
337         prior_history = list(initial.get("search_history", []))
338         original_warm_tmax = float(initial.get("warm_start_Tmax",
339             ↪ initial["Tmax"]))

```

```

256 now = time.monotonic()
257 usable = max(1.0, deadline - now - reserve)
258 remaining_names = process_order[position:]
259 denominator = sum(weights.get(name, 1.0) for name in
260 ↪ remaining_names)
261 budget = max(1.0, usable * weights.get(case_name, 1.0) /
262 ↪ max(denominator, 1e-9))
263 status = "PROVEN_MINIMUM" if n_uav == int(lower["N"]) else
264 ↪ "FEASIBLE_CANDIDATE"
265 print(
266     f"[{case_name}] strict lower bound={lower['N']}, fixed
267     ↪ N={n_uav}, "
268     f"fleet status={status}, search budget={budget:.1f}s,
269     ↪ workers={workers}",
270     flush=True,
271 )
272 case_started = time.monotonic()
273 (obj, routes), _, _, history = solve(
274     case_name,
275     df,
276     n_uav,
277     seeds=4096,
278     time_limit_s=budget,
279     workers=workers,
280     warm_routes=warm_routes,
281     candidate_k=candidate_k,
282     progress=True,
283     return_history=True,
284 )
285 packed = pack_case(case_name, df, routes)
286 packed["theoretical_lower_bound_N"] = int(lower["N"])
287 packed["lower_bound_details"] = lower
288 packed["fleet_size_status"] = status
289 packed["makespan_status"] = "BEST_FOUND_NOT_PROVEN_OPTIMAL"
290 packed["search_history"] = prior_history + history
291 packed["warm_start_Tmax"] = original_warm_tmax
292 packed["improvement_hours"] = float(original_warm_tmax -
293 ↪ packed["Tmax"])
294 improved[case_name] = packed
295 report["cases"][case_name] = {
296     "N": n_uav,
297     "lower_bound": lower,
298     "fleet_size_status": status,
299     "budget_seconds": budget,
300     "actual_seconds": time.monotonic() - case_started,
301     "warm_start_Tmax": original_warm_tmax,
302     "resume_input_Tmax": float(initial["Tmax"]),
303     "best_Tmax": float(packed["Tmax"]),
304     "improvement_hours": float(original_warm_tmax -
305 ↪ packed["Tmax"]),
306     "completed_starts_this_run": len([x for x in history if
307 ↪ x.get("seed") is not None]),
308     "completed_starts": len([x for x in prior_history +
309 ↪ history if x.get("seed") is not None]),
310 }
311 ordered = {name: improved[name] for name in book.sheet_names}
312 validate_all(input_path, ordered, config)
313 report["elapsed_seconds"] = time.monotonic() - started
314 report["validation"] = "PASSED"
315 return ordered, report
316
317 def main() -> None:
318     parser = argparse.ArgumentParser(description="多无人机协同巡检问
319     ↪ 题一求解器")
320     parser.add_argument("--mode", choices=["reproduce", "solve",
321     ↪ "ten-minute"], default="reproduce",
322     help="reproduce 复现; solve 完整搜索;
323     ↪ ten-minute 按统一短时预算并行改进")
324     parser.add_argument("--quality", choices=["fast", "standard",
325     ↪ "thorough"], default="standard")
326     parser.add_argument("--engine", choices=["heuristic", "hybrid",
327     ↪ "exact"], default="hybrid",
328     help="heuristic 仅搜索; hybrid
329     ↪ 搜索失败后精确判定; exact 仅精确 MILP")
330     parser.add_argument("--total-time-limit", type=float,
331     ↪ default=None, metavar="SECONDS",
332     help="ten-minute 模式的统一墙钟预算, 默认读取
333     ↪ config.json")
334     parser.add_argument("--workers", type=int, default=None,
335     help="并行随机起点进程数, 默认读取
336     ↪ config.json")
337     parser.add_argument("--candidate-k", type=int, default=None,
338     help="候选近邻数量, 默认读取 config.json")
339     parser.add_argument("--input", type=Path, default=ROOT / "input"
340     ↪ / "附件1.xlsx")
341     parser.add_argument("--output-dir", type=Path, default=None)
342     parser.add_argument("--resume-from", type=Path, default=None,
343     help="ten-minute 模式从已校验的 solution.json
344     ↪ 继续热启动搜索")
345     args = parser.parse_args()
346     config = load_config()
347     report = None
348     if args.mode == "reproduce":
349         results = json.loads((ROOT / "data" /
350         ↪ "precomputed_solution.json").read_text(encoding="utf-8"))
351     elif args.mode == "solve":
352         results = solve_from_scratch(args.input, args.quality,
353         ↪ args.engine, config)
354     else:
355         total_seconds = float(
356             args.total_time_limit
357             if args.total_time_limit is not None
358             else config.get("ten_minute_total_seconds", 600)
359         )
360         workers = int(args.workers if args.workers is not None else
361         ↪ config.get("parallel_workers", 4))
362         candidate_k = int(
363             args.candidate_k
364             if args.candidate_k is not None
365             else config.get("candidate_neighborhood_size", 24)
366         )
367         if total_seconds <= 30 or workers < 1 or candidate_k < 1:
368             parser.error("ten-minute 参数要求总时间>30 秒、workers>=1、
369             ↪ candidate-k>=1")
370         os.environ["LOKY_MAX_CPU_COUNT"] = str(workers)
371         results, report = run_ten_minute_plan(
372             args.input, config, total_seconds, workers, candidate_k,
373             ↪ args.resume_from
374         )
375     validate_all(args.input, results, config)
376     output_dir = args.output_dir or (ROOT / ("outputs_10min" if
377     ↪ args.mode == "ten-minute" else "outputs"))
378     output_dir.mkdir(parents=True, exist_ok=True)
379     json_path = output_dir / "solution.json"
380     xlsx_path = output_dir / "result1.xlsx"
381     json_path.write_text(json.dumps(results, ensure_ascii=False,
382     ↪ indent=2), encoding="utf-8")
383     export_workbook(results, xlsx_path, config)
384     if report is not None:
385         (output_dir / "search_report.json").write_text(
386             json.dumps(report, ensure_ascii=False, indent=2),
387             ↪ encoding="utf-8"
388         )
389     print(f"Validation passed.\nJSON: {json_path}\nExcel:
390     ↪ {xlsx_path}")
391
392 if __name__ == "__main__":
393     main()

```

问题一启发式求解核心模块

路径: submission_package/question1/program/src/uav_q1/solver_engine.py

```

1 import math, json, os, random, time
2 from concurrent.futures import FIRST_COMPLETED, ProcessPoolExecutor,
3   ↪ wait
4 import numpy as np
5 import pandas as pd
6 from sklearn.cluster import KMeans

```

```

6 from scipy.sparse.csgraph import minimum_spanning_tree
7
8 SPEED=55.0; UNIT_KM=0.1; SERVICE_H=5/60
9 REQ={'I':3,'II':2,'III':1}
10
11 def valid(r):
12     return all(r[i]!=r[i+1] for i in range(len(r)-1))
13
14 def dist(r,p):
15     if not r:return 0.0
16     q=p[r]
17     return np.linalg.norm(q[0])+np.linalg.norm(q[-1])+np.linalg.norm(
18         ↪ (q[1:]-q[-1],axis=1).sum()
19
20 def rtime(r,p): return dist(r,p)*UNIT_KM/SPEED+len(r)*SERVICE_H
21
22 def distance_matrix(p):
23     q=np.vstack([[0.0,0.0],p])
24     return np.linalg.norm(q[:,None,:]-q[None,:,:],axis=2)
25
26 def rtime_matrix(r,D):
27     if not r:return 0.0
28     z=[x+1 for x in r]
29     units=D[0,z[0]]+D[z[-1],0]
30     if len(z)>1:units+=sum(D[a,b] for a,b in zip(z,z[1:]))
31     return float(units)*UNIT_KM/SPEED+len(r)*SERVICE_H
32
33 def candidate_sets(p,k):
34     n=len(p);k=max(1,min(int(k),max(1,n-1)))
35     raw=np.linalg.norm(p[:,None,:]-p[None,:,:],axis=2)
36     return [set(np.argsort(raw[i])[1:k+1].tolist()) for i in
37         ↪ range(n)]
38
39 def removal_time(r,pos,current_time,D):
40     node=r[pos]+1;left=0 if pos==0 else r[pos-1]+1;right=0 if
41         ↪ pos==len(r)-1 else r[pos+1]+1
42     delta=D[left,right]-D[left,node]-D[node,right]
43     return current_time+float(delta)*UNIT_KM/SPEED-SERVICE_H
44
45 def best_insert_fast(r,node,D,near):
46     legal=[];preferred=[]
47     for pos in range(len(r)+1):
48         a=None if pos==0 else r[pos-1];b=None if pos==len(r) else
49             ↪ r[pos]
50         if a==node or b==node:continue
51         ai=0 if a is None else a+1;bi=0 if b is None else
52             ↪ b+1;ni=node+1
53         delta=float(D[ai,ni]+D[ni,bi]-D[ai,bi])
54         item=(delta,pos)
55         legal.append(item)
56         if a is None or b is None or a in near[node] or b in
57             ↪ near[node]:preferred.append(item)
58     pool=preferred or legal
59     return min(pool) if pool else None
60
61 def two_opt(r,p,passes=20):
62     r=list(r);n=len(r)
63     for _ in range(passes):
64         best=0; move=None
65         for i in range(n-1):
66             a=None if i==0 else r[i-1];b=r[i]
67             for j in range(i+1,n):
68                 c=r[j];d=None if j==n-1 else r[j+1]
69                 # reversing preserves internal adjacency; check only
70                 ↪ new boundary pairs
71                 if a is not None and a==c:continue
72                 if d is not None and b==d:continue
73                 old=(np.linalg.norm(p[b])+np.linalg.norm(p[c])
74                     ↪ np.linalg.norm(p[a]-p[b]))+(np.linalg.norm(p[c])
75                     ↪ np.linalg.norm(p[a]-p[b]))+(np.linalg.norm(p[c])
76                     ↪ np.linalg.norm(p[d]))
77                 new=(np.linalg.norm(p[c])+np.linalg.norm(p[b])
78                     ↪ np.linalg.norm(p[a]-p[c]))+(np.linalg.norm(p[b])
79                     ↪ np.linalg.norm(p[a]-p[c]))+(np.linalg.norm(p[b])
80                     ↪ np.linalg.norm(p[d]))
81                 if new<old:best=i+1;move=(i,j)
82             if move is None:break
83         i,j=move;r[i:j+1]=reversed(r[i:j+1])
84     assert valid(r)
85     return r
86
87 def insert_options(r,node,p):
88     out=[]
89     for pos in range(len(r)+1):
90         a=None if pos==0 else r[pos-1];b=None if pos==len(r) else
91             ↪ r[pos]
92         if a==node or b==node:continue
93         old=0 if a is None or b is None else
94             ↪ np.linalg.norm(p[a]-p[b])
95         new=(np.linalg.norm(p[a]-p[node]))+(np.linalg.norm(p[node])
96             ↪ np.linalg.norm(p[b]))
97         if b is None else np.linalg.norm(p[node]-p[b]))
98         out.append((new-old,pos))
99     return out
100
101 def can_remove(r,pos):
102     return not (0<pos<len(r)-1 and r[pos-1]==r[pos+1])
103
104 def init_unique(points,req,k,seed,mode):
105     n=len(points);routes=[] for _ in range(k)
106     if mode=='kmeans':
107         labels=KMeans(n_clusters=k,n_init=10,random_state=seed).fit(
108             ↪ points,sample_weight=req).labels_
109         groups=[np.where(labels==c)[0].tolist() for c in range(k)]
110     else:
111         ang=np.arctan2(points[:,1],points[:,0]);order=np.argsort(ang
112             ↪ ).tolist();shift=seed%n;order=order[shift:]+order[:shift]
113         groups=[[] for _ in range(k)];loads=[0]*k
114         for i in order:
115             c=min(range(k),key=lambda
116                 ↪ z:loads[z]);groups[c].append(i);loads[c]+=req[i]
117     for c,ids in enumerate(groups):
118         if not ids:continue
119         cur=min(ids,key=lambda i:np.linalg.norm(points[i]));routes[c]
120             ↪ =[cur];rem=set(ids);rem.remove(cur)
121         while rem:
122             cur=min(rem,key=lambda j:np.linalg.norm(points[j]-points
123                 ↪ [cur]));routes[c].append(cur);rem.remove(cur)
124         routes[c]=two_opt(routes[c],points)
125     return routes
126
127 def add_extra_visits(routes,points,req,rng):
128     extras=[]
129     for i,r in enumerate(routes):extras+= [i]*(int(r)-1)
130     extras.sort(key=lambda
131         ↪ i:(-np.linalg.norm(points[i]),rng.random()))
132     for node in extras:
133         cur=[rtime(r,points) for r in routes];best=None
134         for k,r in enumerate(routes):
135             for delta,pos in insert_options(r,node,points):
136                 nt=cur[k]+delta*UNIT_KM/SPEED+SERVICE_H
137                 obj=max([cur[z] for z in range(len(routes)) if
138                     ↪ z!=k]+[nt])
139                 val=(obj,nt,delta)
140                 if best is None or val<best[0]:best=(val,k,pos)
141                 if best is None:raise RuntimeError('No valid insertion')
142                 _,k,pos=best;routes[k].insert(pos,node)
143     return [two_opt(r,points) for r in routes]
144
145 def improve(routes,p,rng,iterations,candidate_k=24):
146     routes=[two_opt(r,p) for r in routes];D=distance_matrix(p);near=[
147         ↪ candidate_sets(p,candidate_k)
148     times=np.array([rtime_matrix(r,D) for r in routes])
149     best=[r[0] for r in routes];best_obj=times.max();temp=.06
150     for it in range(iterations):
151         src=int(np.argmax(times)) if rng.random()<.72 else
152             ↪ rng.randrange(len(routes))
153         if not routes[src]:continue
154         pos=rng.randrange(len(routes[src]));node=routes[src][pos]
155         if not can_remove(routes[src],pos):continue
156         nrs=routes[src][:pos]+routes[src][pos+1:];ts=removal_time(ro
157             ↪ utes[src],pos,times[src],D)
158         bestmove=None
159         for dst in range(len(routes)):
160             if dst==src:continue
161             choice=best_insert_fast(routes[dst],node,D,near)
162             if choice is None:continue
163             delta,pos2=choice
164             nrd=routes[dst][:pos2]+[node]+routes[dst][pos2:]
165             td=times[dst]+delta*UNIT_KM/SPEED+SERVICE_H
166             others=[times[z] for z in range(len(routes)) if z not in
167                 ↪ (src,dst)]
168             newmax=max(others+[ts,td]) if others else max(ts,td)

```

```

141 cand=(newmax,dst,pos2,nrd,td)
142 if bestmove is None or cand[0]<bestmove[0]:bestmove=cand
143 if bestmove is None:continue
144 newmax,dst,pos2,nrd,td=bestmove
145 change=newmax-times.max()
146 if change<0 or
    ↪ rng.random()<math.exp(-max(0,change)/max(temp,1e-8)):
147 routes[src],routes[dst]=nrs,nrd;times[src],times[dst]=ts
    ↪ ,td
148 if it%250==0:
149 routes[src]=two_opt(routes[src],p,4);routes[dst]=two
    ↪ _opt(routes[dst],p,4)
150 times[src]=rtime_matrix(routes[src],D);times[dst]=rt
    ↪ ime_matrix(routes[dst],D)
151 if times.max()<best_obj-1e-9:
152 best_obj=times.max();best=[r[:] for r in routes]
153 temp*=-.99975
154 best=[two_opt(r,p,30) for r in best]
155 return polish_balance(best,p,candidate_k=candidate_k)
156
157 def polish_balance(routes,p,rounds=80,candidate_k=24):
158 routes=[r[:] for r in routes];D=distance_matrix(p);near=candidat
    ↪ e_sets(p,candidate_k)
159 for _ in range(rounds):
160 times=[rtime_matrix(r,D) for r in routes];old=max(times);src
    ↪ =int(np.argmax(times));candidate=None
161 # Exhaustive relocate from current longest route.
162 for pos,node in enumerate(routes[src]):
163 if not can_remove(routes[src],pos):continue
164 a=routes[src][:pos]+routes[src][pos+1:];t=removal_time(
    ↪ routes[src],pos,times[src],D)
165 for dst in range(len(routes)):
166 if dst==src:continue
167 choice=best_insert_fast(routes[dst],node,D,near)
168 if choice is None:continue
169 delta,q=choice
170 b=routes[dst][:q]+[node]+routes[dst][q:];tb=times[ds
    ↪ t]+delta*UNIT_KM/SPEED+SERVICE_H
171 nm=max([times[z] for z in range(len(routes)) if z not
    ↪ in (src,dst)]+[ta,tb])
172 if nm<old-1e-9 and (candidate is None or
    ↪ nm<candidate[0]):candidate=(nm,src,dst,a,b)
173 # Exhaustive one-for-one swaps involving current longest
    ↪ route.
174 for dst in range(len(routes)):
175 if dst==src:continue
176 for i,a_node in enumerate(routes[src]):
177 for j,b_node in enumerate(routes[dst]):
178 if a_node==b_node:continue
179 if b_node not in near[a_node] and a_node not in
    ↪ near[b_node]:continue
180 a=routes[src][:i]+b=routes[dst][:j];a[i],b[j]=b_no
    ↪ de,a_node
181 if not valid(a) or not valid(b):continue
182 ta=rtime_matrix(a,D);tb=rtime_matrix(b,D)
183 nm=max([times[z] for z in range(len(routes)) if
    ↪ not in (src,dst)]+[ta,tb])
184 if nm<old-1e-9 and (candidate is None or
    ↪ nm<candidate[0]):candidate=(nm,src,dst,a,b)
185 if candidate is None:break
186 _,src,dst,a,b=candidate
187 routes[src]=two_opt(a,p,8);routes[dst]=two_opt(b,p,8)
188 return routes
189
190 def _progress(label,done,total,best,started,active,time_limit_s=None)
    ↪ ):
191 width=24;elapsed=time.monotonic()-started
192 ratio=elapsed/max(.001,time_limit_s) if time_limit_s is not None
    ↪ else done/max(1,total)
193 filled=min(width,int(width*ratio));bar=' '*filled+'
    ↪ '*(width-filled)
194 best_text='--' if best is None else f'{best:.5f}h'
195 print(f'\r[{label}] [{bar}] {done}/{total} active={active}
    ↪ best={best_text} elapsed={elapsed:6.1f}s',end='',flush=True)
196
197 def _solve_one_seed(payload):
198 cname,df,k,s,warm_routes,candidate_k=payload
199 os.environ.setdefault('OMP_NUM_THREADS','1');os.environ.setdefault
    ↪ ('OPENBLAS_NUM_THREADS','1')
200 p=df[['X_Coordinate','Y_Coordinate']].to_numpy(float);req=df.Ins
    ↪ pction_Level.map(REQ).to_numpy(int)
201 started=time.monotonic();rng=random.Random(9187+s)
202 if warm_routes is not None and s<8:
203 routes=[list(r) for r in warm_routes]
204 else:
205 routes=init_unique(p,req,k,301+s,'kmeans' if s%2==0 else
    ↪ 'sweep')
206 routes=add_extra_visits(routes,p,req,rng)
207 routes=improve(routes,p,random.Random(40001+s),12000+80*len(df),
    ↪ candidate_k=candidate_k)
208 obj=max(rtime(r,p) for r in routes)
209 return {'seed':s,'Tmax':float(obj),'routes':routes,'elapsed_seco
    ↪ nds':time.monotonic()-started}
210
211 def solve(cname,df,k,seeds=14,time_limit_s=None,workers=1,warm_route
    ↪ s=None,
212 candidate_k=24,progress=False,return_history=False):
213 p=df[['X_Coordinate','Y_Coordinate']].to_numpy(float);req=df.Ins
    ↪ pction_Level.map(REQ).to_numpy(int)
214 started=time.monotonic();history=[];ans=None
215 if warm_routes is not None:
216 warm=[list(r) for r in warm_routes]
217 warm_obj=max(rtime(r,p) for r in warm)
218 ans=(warm_obj,warm);history.append({'kind':'warm_start','see
    ↪ d':None,'Tmax':float(warm_obj),'elapsed_seconds':0.0})
219 workers=max(1,int(workers));seeds=max(1,int(seeds));deadline=None
    ↪ if time_limit_s is None else started+float(time_limit_s)
220
221 if workers==1:
222 for s in range(seeds):
223 if deadline is not None and
    ↪ time.monotonic()>=deadline:break
224 item=_solve_one_seed((cname,df,k,s,warm_routes,candidate
    ↪ _k));history.append({'k:v for k,v in item.items() if
    ↪ k!='routes'})
225 if ans is None or
    ↪ item['Tmax']<ans[0]:ans=(item['Tmax'],item['routes'])
226 if progress:_progress(f'{cname} N={k}',len(history)-(1 if
    ↪ warm_routes is not None else
    ↪ 0),seeds,ans[0],started,0,time_limit_s)
227 else:
228 pending={};next_seed=0;done=0
229 with ProcessPoolExecutor(max_workers=workers) as pool:
230 while next_seed<seeds and len(pending)<workers:
231 fut=pool.submit(_solve_one_seed,(cname,df,k,next_see
    ↪ d,warm_routes,candidate_k));pending[fut]=next_se
    ↪ ed;next_seed+=1
232 while pending:
233 completed,_=wait(tuple(pending),timeout=.5,return_wh
    ↪ en=FIRST_COMPLETED)
234 if not completed:
235 if progress:_progress(f'{cname}
    ↪ N={k}',done,seeds,None if ans is None else
    ↪ ans[0],started,len(pending),time_limit_s)
236 continue
237 for fut in completed:
238 pending.pop(fut,None);item=fut.result();done+=1
239 history.append({'key:value for key,value in
    ↪ item.items() if key!='routes'})
240 if ans is None or item['Tmax']<ans[0]:ans=(item[
    ↪ 'Tmax'],item['routes'])
241 if (deadline is None or
    ↪ time.monotonic()<deadline) and
    ↪ next_seed<seeds:
242 nxt=pool.submit(_solve_one_seed,(cname,df,k,
    ↪ next_seed,warm_routes,candidate_k));pend
    ↪ ing[nxt]=next_seed;next_seed+=1
243 if progress:_progress(f'{cname}
    ↪ N={k}',done,seeds,None if ans is None else
    ↪ ans[0],started,len(pending),time_limit_s)
244 if deadline is not None and
    ↪ time.monotonic()>=deadline:
245 next_seed=seeds
246 if progress:print()
247 if ans is None:raise RuntimeError(f'{cname} N={k}: no completed
    ↪ seed and no warm start')

```

```

248 history.sort(key=lambda x:(x.get('seed') is None,-1 if
    ↪ x.get('seed') is None else x['seed']))
249 if return_history:return ans,p,req,history
250 return ans,p,req
251
252 def main():
253     xl=pd.ExcelFile('upload/附件 1.xlsx');out={}
254     for cname in xl.sheet_names:
255         df=pd.read_excel(xl,sheet_name=cname);p=df[['X_Coordinate','
    ↪ Y_Coordinate']].to_numpy(float);req=df.Inspection_Level.
    ↪ map(REQ).to_numpy(int)
256         aug=np.vstack([[0,0],p]);D=np.linalg.norm(aug[:,None]-aug[No
    ↪ ne,:],axis=2)*UNIT_KM
257         mst=float(minimum_spanning_tree(D).sum());lb=math.ceil((req.
    ↪ sum()*SERVICE_H+mst/SPEED)/9-1e-12)
258         print(cname,'tasks',req.sum(),'service',req.sum()*SERVICE_H,
    ↪ 'LB',lb,flush=True)
259         k=max(1,lb)
260         while True:
261             (obj,routes),p,req=solve(cname,df,k)
262             times=[rtime(r,p) for r in routes]
263             print(' k',k,'T',np.round(sorted(times),5),'valid',all(v
    ↪ alid(r) for r in routes),flush=True)
264             if max(times)<=9+1e-9:break
265             k+=1
266             rr=[]
267             for uid,r in enumerate(routes,1):
268                 seq=[int(df.iloc[i].Point_ID) for i in r]
269                 rr.append({'uav':uid,'sequence':[0]+seq+[0],'distance_km'
    ↪ ':dist(r,p)*UNIT_KM','service_count':len(r),'time_h':
    ↪ rtime(r,p)})
270             rr.sort(key=lambda x:x['time_h'],reverse=True)
271             for uid,x in enumerate(rr,1):x['uav']=uid
272             out[cname]={'N':k,'lower_bound_N':lb,'Tmax':max(x['time_h'
    ↪ ' for x in rr),'Tmin':min(x['time_h'] for x in
    ↪ rr),'routes':rr}
273             json.dump(out,open('q1_results_revisit.json','w'),ensure_ascii=F
    ↪ else,indent=2)
274
275 if __name__=='__main__':main()

```

问题一固定机队规模精确模型

路径: submission_package/question1/program/src/uav_q1/exact_milp.py

```

1 """ 固定无人机数量下的精确 MILP 模型。
2
3 该模块不替代快速启发式算法,而是用于区分:
4
5 * OPTIMAL: 已证明固定 N 下的最优最长工作时间;
6 * INFEASIBLE: 已证明固定 N 无法在 9 小时内完成;
7 * FEASIBLE: 时限内得到可行解,但尚未证明最优;
8 * UNKNOWN: 时限内既没有可行解,也没有不可行证明。
9
10 模型把每一次巡检展开成一个任务副本。同一物理点的任务副本之间不建立
11 直接弧,因此同一无人机不能靠原地停留重复计数,但可以在访问其他点后返回。
12 """
13 from __future__ import annotations
14
15 from dataclasses import dataclass
16 from typing import Any
17
18 import numpy as np
19 import pandas as pd
20 from scipy.optimize import Bounds, LinearConstraint, milp
21 from scipy.sparse import coo_matrix
22
23 from .solver_engine import REQ, SERVICE_H, SPEED, UNIT_KM, rtime
24
25
26 @dataclass
27 class ExactResult:
28     status: str
29     message: str
30     routes: list[list[int]] | None
31     objective_h: float | None
32     dual_bound_h: float | None
33     mip_gap: float | None
34     node_count: int | None
35
36 class _Rows:
37     """ 稀疏线性约束构造器。"""
38
39     def __init__(self) -> None:
40         self.row: list[int] = []
41         self.col: list[int] = []
42         self.data: list[float] = []
43         self.lb: list[float] = []
44         self.ub: list[float] = []
45
46     def add(self, terms: dict[int, float], lb: float, ub: float) ->
    ↪ None:
47         rid = len(self.lb)
48         for col, value in terms.items():
49             if abs(value) > 1e-15:
50                 self.row.append(rid)
51                 self.col.append(col)
52                 self.data.append(float(value))
53                 self.lb.append(float(lb))
54                 self.ub.append(float(ub))
55
56     def constraint(self, nvar: int) -> LinearConstraint:
57         matrix = coo_matrix(
58             (self.data, (self.row, self.col)),
59             shape=(len(self.lb), nvar),
60             ).tocsr()
61         return LinearConstraint(matrix, np.asarray(self.lb),
    ↪ np.asarray(self.ub))
62
63 def _task_copies(df: pd.DataFrame) -> tuple[np.ndarray, np.ndarray]:
64     """ 返回每个任务副本对应的物理行号和坐标。"""
65     physical: list[int] = []
66     for i, level in enumerate(df["Inspection_Level"]):
67         physical.extend([i] * REQ[str(level)])
68     physical_arr = np.asarray(physical, dtype=int)
69     points = df[["X_Coordinate", "Y_Coordinate"]].to_numpy(float)
70     return physical_arr, points[physical_arr]
71
72 def solve_fixed_n_exact(
73     df: pd.DataFrame,
74     n_uav: int,
75     limit_h: float = 9.0,
76     time_limit_s: float = 300.0,
77     mip_rel_gap: float = 0.0,
78 ) -> ExactResult:
79     """ 用完整弧集 MILP 求固定 ``n_uav`` 的 min-max 路径。
80
81     SciPy/HiGHS 返回 status=2 时才记为严格不可行。达到时限且没有解时记为
82     UNKNOWN, 程序不得据此排除当前 N。
83
84     """
85     physical, task_xy = _task_copies(df)
86     m = len(physical)
87     if n_uav < 1 or n_uav > m:
88         raise ValueError("n_uav 必须位于 1 和任务副本总数之间")
89
90     # 节点 0 为基地; 1..m 为任务副本。相同物理点的副本之间禁止直接转移。
91     xy = np.vstack([[0.0, 0.0], task_xy])
92     arcs: list[tuple[int, int]] = []
93     for i in range(m + 1):
94         for j in range(m + 1):
95             if i == j or (i == 0 and j == 0):
96                 continue
97             if i > 0 and j > 0 and physical[i - 1] == physical[j -
    ↪ 1]:
98                 continue
99             arcs.append((i, j))
100
101     a_count = len(arcs)
102     out_arcs: list[list[int]] = [[] for _ in range(m + 1)]
103     in_arcs: list[list[int]] = [[] for _ in range(m + 1)]
104     for a, (i, j) in enumerate(arcs):
105         out_arcs[i].append(a)

```

```

107         in_arcs[j].append(a)
108
109     # 变量块: [x 二元弧][f 连续单商品流][y 二元分配][Cmax]。
110     x0 = 0
111     f0 = n_uav * a_count
112     y0 = f0 + n_uav * a_count
113     c_idx = y0 + n_uav * m
114     nvar = c_idx + 1
115
116     def xid(k: int, a: int) -> int:
117         return x0 + k * a_count + a
118
119     def fid(k: int, a: int) -> int:
120         return f0 + k * a_count + a
121
122     def yid(k: int, i: int) -> int:
123         # i 是 1..m 的任务节点号。
124         return y0 + k * m + (i - 1)
125
126     c = np.zeros(nvar)
127     c[c_idx] = 1.0
128     integrality = np.zeros(nvar, dtype=np.uint8)
129     integrality[x0:f0] = 1
130     integrality[y0:c_idx] = 1
131     lower = np.zeros(nvar)
132     upper = np.full(nvar, np.inf)
133     upper[x0:f0] = 1.0
134     upper[f0:y0] = float(m)
135     upper[y0:c_idx] = 1.0
136     upper[c_idx] = float(limit_h)
137
138     rows = _Rows()
139
140     # 每个任务副本恰由一架无人机完成。
141     for i in range(1, m + 1):
142         rows.add({xid(k, i): 1.0 for k in range(n_uav)}, 1.0, 1.0)
143
144     # 每架无人机都从基地出发一次并返回一次; 任务点入度=出度=分配变量。
145     for k in range(n_uav):
146         rows.add({xid(k, a): 1.0 for a in out_arcs[0]}, 1.0, 1.0)
147         rows.add({xid(k, a): 1.0 for a in in_arcs[0]}, 1.0, 1.0)
148         for i in range(1, m + 1):
149             out_terms = {xid(k, a): 1.0 for a in out_arcs[i]}
150             out_terms[yid(k, i)] = -1.0
151             rows.add(out_terms, 0.0, 0.0)
152             in_terms = {xid(k, a): 1.0 for a in in_arcs[i]}
153             in_terms[yid(k, i)] = -1.0
154             rows.add(in_terms, 0.0, 0.0)
155
156     # 单商品流消除不经过基地的子回路。
157     for k in range(n_uav):
158         depot_terms: dict[int, float] = {}
159         for a in out_arcs[0]:
160             depot_terms[fid(k, a)] = depot_terms.get(fid(k, a), 0.0)
161             ↪ + 1.0
162         for a in in_arcs[0]:
163             depot_terms[fid(k, a)] = depot_terms.get(fid(k, a), 0.0)
164             ↪ - 1.0
165         for i in range(1, m + 1):
166             depot_terms[yid(k, i)] = -1.0
167         rows.add(depot_terms, 0.0, 0.0)
168
169     for i in range(1, m + 1):
170         terms: dict[int, float] = {yid(k, i): -1.0}
171         for a in in_arcs[i]:
172             terms[fid(k, a)] = terms.get(fid(k, a), 0.0) + 1.0
173         for a in out_arcs[i]:
174             terms[fid(k, a)] = terms.get(fid(k, a), 0.0) - 1.0
175         rows.add(terms, 0.0, 0.0)
176
177     for a in range(a_count):
178         rows.add({fid(k, a): 1.0, xid(k, a): -float(m)}, -np.inf,
179                 ↪ 0.0)
180
181     arc_time = np.asarray([
182         np.linalg.norm(xy[i] - xy[j]) * UNIT_KM / SPEED for i, j in
183         ↪ arcs
184     ])
185
186     def time_terms(k: int) -> dict[int, float]:
187         terms = {xid(k, a): float(arc_time[a]) for a in
188                 ↪ range(a_count)}
189
190     for i in range(1, m + 1):
191         terms[yid(k, i)] = SERVICE_H
192     return terms
193
194 # Cmax 上界已经固定为 9 小时; 同时以 Cmax 为第二阶段目标。
195 all_time_terms = [time_terms(k) for k in range(n_uav)]
196 for terms in all_time_terms:
197     terms = dict(terms)
198     terms[c_idx] = -1.0
199     rows.add(terms, -np.inf, 0.0)
200
201 # 对同质无人机按工作时长降序编号, 削减车辆置换对称性。
202 for k in range(n_uav - 1):
203     terms = dict(all_time_terms[k + 1])
204     for idx, value in all_time_terms[k].items():
205         terms[idx] = terms.get(idx, 0.0) - value
206     rows.add(terms, -np.inf, 0.0)
207
208 # 同一物理点的副本无差别: 按所分配的无人机编号排序, 削减副本对称性。
209 for p in np.unique(physical):
210     copies = np.flatnonzero(physical == p) + 1
211     for left, right in zip(copies[:-1], copies[1:]):
212         terms: dict[int, float] = {}
213         for k in range(n_uav):
214             terms[yid(k, int(left))] = float(k)
215             terms[yid(k, int(right))] = -float(k)
216         rows.add(terms, -np.inf, 0.0)
217
218 result = milp(
219     c=c,
220     integrality=integrality,
221     bounds=Bounds(lower, upper),
222     constraints=rows.constraint(nvar),
223     options={
224         "time_limit": float(time_limit_s),
225         "mip_rel_gap": float(mip_rel_gap),
226         "presolve": True,
227     },
228 )
229
230 status_map = {0: "OPTIMAL", 1: "LIMIT", 2: "INFEASIBLE", 3:
231 ↪ "UNBOUNDED", 4: "ERROR"}
232 raw_status = status_map.get(int(result.status), "ERROR")
233 if raw_status == "INFEASIBLE":
234     return ExactResult("INFEASIBLE", result.message, None, None,
235                        getattr(result, "mip_dual_bound", None),
236                        getattr(result, "mip_gap", None),
237                        getattr(result, "mip_node_count", None))
238
239 if result.x is None:
240     return ExactResult("UNKNOWN", result.message, None, None,
241                        getattr(result, "mip_dual_bound", None),
242                        getattr(result, "mip_gap", None),
243                        getattr(result, "mip_node_count", None))
244
245 vector = np.asarray(result.x)
246 routes: list[list[int]] = []
247 for k in range(n_uav):
248     route: list[int] = []
249     current = 0
250     for _ in range(m + 1):
251         chosen = [a for a in out_arcs[current] if vector[xid(k,
252 ↪ a)] > 0.5]
253         if len(chosen) != 1:
254             return ExactResult("UNKNOWN", "MILP
255 ↪ 解的弧结构无法提取为单一路线", None,
256                                None, getattr(result,
257                                "mip_dual_bound", None),
258                                getattr(result, "mip_gap", None),
259                                getattr(result, "mip_node_count",
260                                ↪ None))
261         _, nxt = arcs[chosen[0]]
262         if nxt == 0:
263             break
264         route.append(int(physical[nxt - 1]))
265         current = nxt
266     else:
267         return ExactResult("UNKNOWN", "路线提取超过任务节点数",
268 ↪ None, None,
269                                getattr(result, "mip_dual_bound",
270                                ↪ None),
271                                getattr(result, "mip_gap", None),

```



```

258         getattr(result, "mip_node_count",
259                 ↳ None))
260     routes.append(route)
261     physical_points = df[["X_Coordinate",
262     ↳ "Y_Coordinate"]].to_numpy(float)
263     objective = max(runtime(route, physical_points) for route in
264     ↳ routes)
265     final_status = "OPTIMAL" if raw_status == "OPTIMAL" else
266     ↳ "FEASIBLE"
267     return ExactResult(final_status, result.message, routes,
268     ↳ objective,
269
270     getattr(result, "mip_dual_bound", None),
271     getattr(result, "mip_gap", None),
272     getattr(result, "mip_node_count", None))
273
274 def result_as_dict(result: ExactResult) -> dict[str, Any]:
275     def number(value: Any) -> float | int | None:
276         if value is None:
277             return None
278         if isinstance(value, (int, np.integer)):
279             return int(value)
280         return float(value)
281
282     return {
283         "status": result.status,
284         "message": result.message,
285         "objective_h": number(result.objective_h),
286         "dual_bound_h": number(result.dual_bound_h),
287         "mip_gap": number(result.mip_gap),
288         "node_count": number(result.node_count),
289     }

```

问题一独立结果校验模块

路径: submission_package/question1/program/src/uav_q1/validation.py

```

1 from __future__ import annotations
2
3 from collections import Counter
4 from pathlib import Path
5 import math
6 import pandas as pd
7
8
9 def validate_all(input_path: Path, results: dict, config: dict) ->
10 ↳ None:
11     required_map = config["level_required_visits"]
12     speed = float(config["speed_kmh"])
13     service_h = float(config["service_minutes_per_visit"]) / 60.0
14     limit_h = float(config["maximum_work_hours"])
15     book = pd.ExcelFile(input_path)
16     assert set(results) == set(book.sheet_names), "结果中的 Case
17     ↳ 与附件 1 不一致"
18
19     for case_name in book.sheet_names:
20         df = pd.read_excel(book, sheet_name=case_name)
21         expected = {int(row.Point_ID):
22         ↳ int(required_map[row.Inspection_Level]) for _, row in
23         ↳ df.iterrows()}
24         coordinates = {int(row.Point_ID): (float(row.X_Coordinate),
25         ↳ float(row.Y_Coordinate)) for _, row in df.iterrows()}
26         found = Counter()
27         case = results[case_name]
28         assert int(case["N"]) == len(case["routes"]), f"{case_name}:
29         ↳ N 与路线行数不符"
30
31         recalculated_times = []
32         for route in case["routes"]:
33             sequence = [int(x) for x in route["sequence"]]
34             assert sequence[0] == sequence[-1] == 0, f"{case_name}:
35             ↳ 路线必须以基地 0 为首尾"
36             tasks = sequence[1:-1]

```

```

30     assert all(tasks[i] != tasks[i + 1] for i in
31     ↳ range(len(tasks) - 1)), \
32         f"{case_name}: 同一点连续出现, 未满足离开后返回规则"
33     found.update(tasks)
34     xy = [(0.0, 0.0), *[coordinates[x] for x in tasks], (0.0,
35     ↳ 0.0)]
36     coordinate_distance = sum(math.dist(xy[i], xy[i + 1]) for
37     ↳ i in range(len(xy) - 1))
38     distance_km = coordinate_distance *
39     ↳ float(config["coordinate_unit_km"])
40     work_h = distance_km / speed + len(tasks) * service_h
41     assert abs(work_h - float(route["time_h"])) < 1e-6,
42     ↳ f"{case_name}: 工作时间计算不一致"
43     assert work_h <= limit_h + 1e-8, f"{case_name}: 存在超过 9
44     ↳ h 的路线"
45     recalculated_times.append(work_h)
46
47     assert dict(found) == expected, f"{case_name}: 巡检次数不满足
48     ↳ I/II/III=3/2/1"
49     assert abs(max(recalculated_times) - float(case["Tmax"])) <
50     ↳ 1e-6
51     assert abs(min(recalculated_times) - float(case["Tmin"])) <
52     ↳ 1e-6

```

问题一工作簿输出模块

路径: submission_package/question1/program/src/uav_q1/excel_output.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4 from openpyxl import Workbook
5 from openpyxl.styles import Alignment, Border, Font, PatternFill,
6 ↳ Side
7 from openpyxl.utils import get_column_letter
8
9 NAVY = "17365D"
10 BLUE = "4472C4"
11 LIGHT = "D9EAF7"
12 WHITE = "FFFFFF"
13 THIN = Side(style="thin", color="D9E2F3")
14
15
16 def style_header(cells) -> None:
17     for cell in cells:
18         cell.fill = PatternFill("solid", fgColor=BLUE)
19         cell.font = Font(bold=True, color=WHITE)
20         cell.alignment = Alignment(horizontal="center",
21         ↳ vertical="center", wrap_text=True)
22         cell.border = Border(left=THIN, right=THIN, top=THIN,
23         ↳ bottom=THIN)
24
25 def export_workbook(results: dict, output_path: Path, config: dict)
26 ↳ -> None:
27     wb = Workbook()
28     ws = wb.active
29     ws.title = "Summary"
30     ws.sheet_view.showGridLines = False
31     ws.merge_cells("A1:D1")
32     ws["A1"] = " 问题一: 多无人机协同巡检结果"
33     ws["A1"].fill = PatternFill("solid", fgColor=NAVY)
34     ws["A1"].font = Font(bold=True, color=WHITE, size=16)
35     ws["A1"].alignment = Alignment(horizontal="center")
36     headers = ["测试算例", "无人机数量 N", "最长工作时间 Tmax (h)",
37     ↳ "最短工作时间 Tmin (h)"]
38     for col, value in enumerate(headers, 1):
39         ws.cell(3, col, value)
40     style_header(ws[3])
41     for row, (case_name, case) in enumerate(results.items(), 4):
42         ws.cell(row, 1, case_name)
43         ws.cell(row, 2, case["N"])
44         ws.cell(row, 3, case["Tmax"])
45         ws.cell(row, 4, case["Tmin"])

```

```

43     ws.cell(row, 3).number_format = ws.cell(row, 4).number_format
44     ↪ = "0.0000"
45 ws["A10"] = " 最终计数规则"
46 ws["B10"] = "到达并作业 5 min 计 1 次; 原地停留不重复计数;
47 ↪ 离开后返回可再次计数; 不同无人机可同时巡检。"
48 ws.merge_cells("B10:D11")
49 ws["A12"] = " 时间公式"
50 ws["B12"] = "T = Flight Distance / 55 + Inspection Count × 5 /
51 ↪ 60"
52 ws.merge_cells("B12:D12")
53 for col, width in zip("ABCD", [20, 25, 30, 30]):
54     ws.column_dimensions[col].width = width
55 ws.freeze_panes = "A4"
56
57 for case_name, case in results.items():
58     sh = wb.create_sheet(case_name)
59     sh.sheet_view.showGridLines = False
60     max_len = max(len(x["sequence"]) for x in case["routes"])
61     headers = ["UAV ID", *[f"{i}th Inspection Point" for i in
62 ↪ range(1, max_len + 1)],
63               "Flight Distance (km)", "Inspection Count",
64 ↪ "Working Time (h)"]
65     for col, value in enumerate(headers, 1):
66         sh.cell(5, col, value)
67     style_header(sh[5])
68     sh["A1"] = f"{case_name} 详细调度方案 (0 表示基地)"
69     end_col = get_column_letter(len(headers))
70     sh.merge_cells(f"A1:{end_col}1")
71     sh["A1"].fill = PatternFill("solid", fgColor=NAVY)
72     sh["A1"].font = Font(bold=True, color=WHITE, size=14)
73     sh["A1"].alignment = Alignment(horizontal="center")
74     sh["A2"] = "N"; sh["B2"] = case["N"]
75     sh["C2"] = "Tmax (h)"; sh["D2"] = case["Tmax"]
76     sh["E2"] = "Tmin (h)"; sh["F2"] = case["Tmin"]
77     for row, route in enumerate(case["routes"], 6):
78         values = [route["uav"], *route["sequence"], *([None] *
79 ↪ (max_len - len(route["sequence"])))],
80                 route["distance_km"], route["service_count"]]
81         for col, value in enumerate(values, 1): sh.cell(row, col,
82 ↪ value)
83         distance_col = 2 + max_len
84         count_col = distance_col + 1
85         time_col = distance_col + 2
86         sh.cell(row, time_col,
87 ↪ f"={get_column_letter(distance_col)}{row}/55+{get_co |
88 ↪ lumn_letter(count_col)}{row}*5/60")
89         sh.cell(row, distance_col).number_format = "0.000"
90         sh.cell(row, time_col).number_format = "0.000000"
91     sh.freeze_panes = "B6"
92     sh.column_dimensions["A"].width = 10
93     for c in range(2, 2 + max_len):
94         ↪ sh.column_dimensions[get_column_letter(c)].width = 11
95     for c in range(2 + max_len, len(headers) + 1):
96         ↪ sh.column_dimensions[get_column_letter(c)].width = 19
97     sh.auto_filter.ref = f"A5:{end_col}{5 + len(case['routes'])}"
98
99 output_path.parent.mkdir(parents=True, exist_ok=True)
100 wb.save(output_path)

```

附录 C 问题二源程序

问题二求解、结果复算及 CSV 输出均集中在同一程序中；用于生成论文图件和最终 Excel 版式的辅助脚本仅放入电子支撑材料。

问题二负载均衡求解与校验程序

路径: submission_package/question2/program/q2_solver.py

```

1 """ 问题二: 固定问题一无人机数量与完成时间上界的负载均衡优化。
2
3 Python 求解器只输出 JSON/CSV; result2.xlsx 由 build_result2.mjs
  ↳ 根据同一 JSON 生成。
4 工作时间仅包含实际飞行与有效巡检服务, 不允许用等待时间改善均衡指标。
5 """
6
7 from __future__ import annotations
8
9 import argparse
10 import csv
11 import json
12 import math
13 import random
14 import time
15 from collections import Counter
16 from dataclasses import dataclass
17 from pathlib import Path
18
19 import numpy as np
20 import pandas as pd
21
22
23 SPEED_KMH = 55.0
24 UNIT_KM = 0.1
25 SERVICE_H = 5.0 / 60.0
26 REQ = {"I": 3, "II": 2, "III": 1}
27 TOL = 1.0e-9
28
29
30 @dataclass(frozen=True)
31 class CaseData:
32     point_ids: tuple[int, ...]
33     coordinates: dict[int, tuple[float, float]]
34     expected_visits: dict[int, int]
35     distance_km: dict[tuple[int, int], float]
36
37
38 def load_case(book: pd.ExcelFile, case_name: str) -> CaseData:
39     df = pd.read_excel(book, sheet_name=case_name)
40     ids = tuple(int(x) for x in df["Point_ID"])
41     coordinates = {
42         int(row.Point_ID): (float(row.X_Coordinate),
43                             ↳ float(row.Y_Coordinate))
44     }
45     for _, row in df.iterrows()
46     }
47     coordinates[0] = (0.0, 0.0)
48     expected = {
49         int(row.Point_ID): REQ[str(row.Inspection_Level)]
50     }
51     for _, row in df.iterrows()
52     }
53     distance: dict[tuple[int, int], float] = {}
54     for i in (0,) + ids:
55         xi, yi = coordinates[i]
56         for j in (0,) + ids:
57             xj, yj = coordinates[j]
58             distance[(i, j)] = math.hypot(xi - xj, yi - yj) * UNIT_KM
59     return CaseData(ids, coordinates, expected, distance)
60
61
62 def route_valid(route: list[int]) -> bool:
63     return bool(route) and all(a != b for a, b in zip(route,
64     ↳ route[1:]))
65
66
67 def route_metrics(route: list[int], data: CaseData) -> tuple[float,
68     ↳ float]:
69     seq = [0] + route + [0]
70     distance = sum(data.distance_km[(a, b)] for a, b in zip(seq,
71     ↳ seq[1:]))
72     return distance, distance / SPEED_KMH + len(route) * SERVICE_H
73
74
75 def all_metrics(routes: list[list[int]], data: CaseData) ->
76     ↳ tuple[list[float], list[float]]:
77     pairs = [route_metrics(route, data) for route in routes]
78     return [x[0] for x in pairs], [x[1] for x in pairs]
79
80
81 def objective(times_h: list[float]) -> tuple[float, float, float]:
82     return max(times_h) - min(times_h), max(times_h), sum(times_h)
83
84
85 def better(a: tuple[float, float, float], b: tuple[float, float,
86     ↳ float]) -> bool:
87     for x, y in zip(a, b):
88         if x < y - TOL:
89             return True
90         if x > y + TOL:
91             return False
92     return False
93
94
95 def two_opt(route: list[int], data: CaseData, max_passes: int = 2) ->
96     ↳ list[int]:
97     """ 仅执行缩短飞行距离的合法 2-opt, 不会人为延长轻载路线。"""
98     best = route[:]
99     if len(best) < 4:
100         return best
101     for _ in range(max_passes):
102         changed = False
103         for i in range(len(best) - 2):
104             a = 0 if i == 0 else best[i - 1]
105             b = best[i]
106             for j in range(i + 1, len(best)):
107                 c = best[j]
108                 d = 0 if j == len(best) - 1 else best[j + 1]
109                 old = data.distance_km[(a, b)] + data.distance_km[(c,
110                 ↳ d)]
111                 new = data.distance_km[(a, c)] + data.distance_km[(b,
112                 ↳ d)]
113                 if new >= old - 1.0e-12:
114                     continue
115                 candidate = best[:i] + list(reversed(best[i : j +
116                 ↳ 1])) + best[j + 1 :]
117                 if route_valid(candidate):
118                     best = candidate
119                     changed = True
120                     break
121             if changed:
122                 break
123         if not changed:
124             break
125     return best
126
127
128 def normalize_changed(
129     routes: list[list[int]], data: CaseData, changed: tuple[int, ...]
130 ) -> list[list[int]]:
131     result = [r[:] for r in routes]
132     for idx in set(changed):
133         result[idx] = two_opt(result[idx], data, max_passes=2)
134     return result
135
136
137 def legal_remove(route: list[int], pos: int) -> bool:

```

```

126 if len(route) <= 1:
127     return False
128 candidate = route[pos] + route[pos + 1 :]
129 return route_valid(candidate)
130
131
132 def legal_insert(route: list[int], node: int, pos: int) -> bool:
133     left = None if pos == 0 else route[pos - 1]
134     right = None if pos == len(route) else route[pos]
135     return left != node and right != node
136
137
138 def choose_heavy_light(times_h: list[float], rng: random.Random) ->
    ↳ tuple[int, int]:
139     order = sorted(range(len(times_h)), key=times_h.__getitem__)
140     width = min(2, max(1, len(order) // 2))
141     if rng.random() < 0.85:
142         src = rng.choice(order[-width:])
143         dst = rng.choice(order[:width])
144     else:
145         src, dst = rng.sample(order, 2)
146     return src, dst
147
148
149 def propose(
150     routes: list[list[int]], data: CaseData, rng: random.Random
151 ) -> list[list[int]] | None:
152     _, times = all_metrics(routes, data)
153     src, dst = choose_heavy_light(times, rng)
154     move = rng.random()
155     candidate = [r[:] for r in routes]
156
157     if move < 0.60:
158         positions = [i for i in range(len(candidate[src])) if
    ↳ legal_remove(candidate[src], i)]
159         if not positions:
160             return None
161         pos = rng.choice(positions)
162         node = candidate[src].pop(pos)
163         legal = [q for q in range(len(candidate[dst]) + 1) if
    ↳ legal_insert(candidate[dst], node, q)]
164         if not legal:
165             return None
166         # 在合法位置中优先选择飞行距离增量较小的几个, 再保留随机性。
167         ranked = []
168         for q in legal:
169             left = 0 if q == 0 else candidate[dst][q - 1]
170             right = 0 if q == len(candidate[dst]) else
    ↳ candidate[dst][q]
171             inc = (
172                 data.distance_km[(left, node)]
173                 + data.distance_km[(node, right)]
174                 - data.distance_km[(left, right)]
175             )
176             ranked.append((inc, q))
177         ranked.sort()
178         q = rng.choice(ranked[: min(4, len(ranked))])
179         candidate[dst].insert(q, node)
180         changed = (src, dst)
181     else:
182         i = rng.randrange(len(candidate[src]))
183         j = rng.randrange(len(candidate[dst]))
184         candidate[src][i], candidate[dst][j] = candidate[dst][j],
    ↳ candidate[src][i]
185         if not route_valid(candidate[src]) or not
    ↳ route_valid(candidate[dst]):
186             return None
187         changed = (src, dst)
188
189     candidate = normalize_changed(candidate, data, changed)
190     if not all(route_valid(r) for r in candidate):
191         return None
192     return candidate
193
194
195 def anneal_seed(
196     initial: list[list[int]],
197     data: CaseData,
198     cap_h: float,
199     seed: int,
200     iterations: int,
201     deadline: float,
202 ) -> tuple[list[list[int]], dict]:
203     rng = random.Random(seed)
204     current = [two_opt(r, data, max_passes=20) for r in initial]
205     _, current_times = all_metrics(current, data)
206     current_key = objective(current_times)
207     if current_key[1] > cap_h + TOL:
208         # 问题一初始路线必须始终可作为保底方案。
209         current = [r[:] for r in initial]
210         _, current_times = all_metrics(current, data)
211         current_key = objective(current_times)
212     best = [r[:] for r in current]
213     best_key = current_key
214     accepted = 0
215     attempted = 0
216
217     for iteration in range(iterations):
218         if time.monotonic() >= deadline:
219             break
220         candidate = propose(current, data, rng)
221         if candidate is None:
222             continue
223         attempted += 1
224         _, times = all_metrics(candidate, data)
225         key = objective(times)
226         if key[1] > cap_h + TOL or key[1] > 9.0 + TOL:
227             continue
228
229         accept = better(key, current_key)
230         if not accept:
231             progress = iteration / max(1, iterations - 1)
232             temperature = 0.02 * (1.0 - progress) + 0.00015
233             loss = max(0.0, key[0] - current_key[0])
234             loss += 0.20 * max(0.0, key[1] - current_key[1])
235             loss += 0.002 * max(0.0, key[2] - current_key[2])
236             accept = rng.random() < math.exp(-loss / temperature)
237         if accept:
238             current = candidate
239             current_key = key
240             accepted += 1
241             if better(key, best_key):
242                 best = [r[:] for r in candidate]
243                 best_key = key
244
245     return best, {
246         "seed": seed,
247         "attempted_moves": attempted,
248         "accepted_moves": accepted,
249         "best_delta_h": best_key[0],
250         "best_Tmax_h": best_key[1],
251         "best_total_h": best_key[2],
252     }
253
254
255 def deterministic_polish(
256     initial: list[list[int]], data: CaseData, cap_h: float,
    ↳ max_rounds: int = 80
257 ) -> list[list[int]]:
258     """ 系统枚举重载到轻载的单任务迁移, 直到没有词典序改善。"""
259     routes = [r[:] for r in initial]
260     for _ in range(max_rounds):
261         _, times = all_metrics(routes, data)
262         old_key = objective(times)
263         order = sorted(range(len(routes)), key=times.__getitem__)
264         sources = order[-min(2, len(order)) :]
265         targets = order[: min(2, len(order))]
266         best_candidate = None
267         best_key = old_key
268         for src in sources:
269             for dst in targets:
270                 if src == dst:
271                     continue
272                 for i, node in enumerate(routes[src]):
273                     if not legal_remove(routes[src], i):
274                         continue
275                     for q in range(len(routes[dst]) + 1):
276                         if not legal_insert(routes[dst], node, q):
277                             continue
278                     candidate = [r[:] for r in routes]
279                     candidate[src].pop(i)
280                     candidate[dst].insert(q, node)

```

```

281         candidate = normalize_changed(candidate, 352         "distance_km": distance,
    ↪ data, (src, dst)) 353         "service_count": len(route),
282         _, cand_times = all_metrics(candidate, data) 354         "time_h": work_h,
283         key = objective(cand_times) 355         "waiting_time_h": 0.0,
284         if key[1] <= cap_h + TOL and key[1] <= 9.0 + 356     }
    ↪ TOL and better(key, best_key): 357 )
285         best_candidate = candidate 358 return {
286         best_key = key 359     "N": int(q1_case["N"]),
287     if best_candidate is None: 360     "q1_Tmax_h": float(q1_case["Tmax"]),
288         break 361     "q1_Tmin_h": float(q1_case["Tmin"]),
289     routes = best_candidate 362     "q1_delta_h": delta1,
290     # 最终把每条路线充分缩短, 避免通过保留可消除的绕行来抬高 Tmin. 363     "epsilon_h": epsilon_h,
291     return [two_opt(route, data, max_passes=100) for route in routes] 364     "completion_time_cap_h": cap,
292     365     "Tmax": max(times),
293     366     "Tmin": min(times),
294 def validate_case( 367     "delta": delta2,
295     case_name: str, 368     "total_work_h": sum(times),
296     routes: list[list[int]], 369     "delta_reduction_h": delta1 - delta2,
297     data: CaseData, 370     "delta_reduction_pct": 0.0 if delta1 <= TOL else (delta1 -
298     q1_case: dict, 371     ↪ delta2) / delta1,
299     epsilon_h: float, 372     "objective_order": ["delta", "Tmax", "total_work_h"],
300 ) -> tuple[list[float], list[float]]: 373     "waiting_allowed": False,
301     assert len(routes) == int(q1_case["N"]), f"{case_name}: 374     "routes": route_records,
    ↪ 无人机数量发生变化" 375     "search_history": history,
302     assert all(route_valid(r) for r in routes), f"{case_name}: 376     "validation": "PASSED",
    ↪ 存在空路线或连续相同巡检点" 377 }
303     actual = Counter(x for route in routes for x in route) 378
304     assert actual == Counter(data.expected_visits), f"{case_name}: 379 def write_csv_outputs(results: dict, output_dir: Path) -> None:
    ↪ 巡检次数不一致" 380     output_dir.mkdir(parents=True, exist_ok=True)
305     distances, times = all_metrics(routes, data) 381     with (output_dir / "summary.csv").open("w", newline="",
306     cap = float(q1_case["Tmax"]) + epsilon_h 382     ↪ encoding="utf-8-sig") as f:
307     assert max(times) <= cap + 1.0e-8, f"{case_name}: 383     w = csv.writer(f)
    ↪ 超过问题一完成时间上界" 384     w.writerow(["Case", "N", "Tmax_h", "Tmin_h", "delta_h",
308     assert max(times) <= 9.0 + 1.0e-8, f"{case_name}: 超过 9 小时" 385     ↪ "total_work_h"])
309     return distances, times 386     for case_name, result in results.items():
310     387     w.writerow([case_name, result["N"], result["Tmax"],
311     388     ↪ result["Tmin"], result["delta"],
312 def solve_case( 389     ↪ result["total_work_h"]])
313     case_name: str, 390     with (output_dir / "comparison.csv").open("w", newline="",
314     data: CaseData, 391     ↪ encoding="utf-8-sig") as f:
315     q1_case: dict, 392     w = csv.writer(f)
316     epsilon_h: float, 393     w.writerow(["Case", "q1_delta_h", "q2_delta_h",
317     seeds: int, 394     ↪ "reduction_h", "reduction_pct"])
318     iterations: int, 395     for case_name, result in results.items():
319     time_limit_s: float, 396     w.writerow([case_name, result["q1_delta_h"],
320 ) -> dict: 397     ↪ result["delta"], result["delta_reduction_h"],
321     initial = [list(map(int, route["sequence"][1:-1])) for route in 398     ↪ result["delta_reduction_pct"]])
    ↪ q1_case["routes"]] 399     for case_name, result in results.items():
322     validate_case(case_name, initial, data, q1_case, epsilon_h) 400     max_len = max(len(route["sequence"]) for route in
323     cap = float(q1_case["Tmax"]) + epsilon_h 401     ↪ result["routes"])
324     deadline = time.monotonic() + time_limit_s 402     headers = ["UAV_ID"] + [f"Stop_{i}" for i in range(1, max_len
325     best = [r[:] for r in initial] 403     ↪ + 1)] + ["Distance_km", "Service_Count",
326     _, best_times = all_metrics(best, data) 404     ↪ "Working_Time_h"]
327     best_key = objective(best_times) 405     with (output_dir / f"{case_name}_routes.csv").open("w",
328     history = [] 406     ↪ newline="", encoding="utf-8-sig") as f:
329     407     w = csv.writer(f)
330     for offset in range(seeds): 408     w.writerow(headers)
331     if time.monotonic() >= deadline: 409     for route in result["routes"]:
332         break 410         padding = [""] * (max_len - len(route["sequence"]))
333     routes, record = anneal_seed( 411     w.writerow([route["uav"], *route["sequence"],
334     initial, data, cap, 20260818 + offset, iterations, 412     ↪ *padding, route["distance_km"],
    ↪ deadline 413     ↪ route["service_count"], route["time_h"]])
335 ) 414
336     history.append(record) 415
337     _, times = all_metrics(routes, data) 416
338     key = objective(times) 417
339     if better(key, best_key): 418     def validate_saved(input_path: Path, q1_path: Path, q2_path: Path) ->
340         best, best_key = routes, key 419     ↪ None:
341     420
342     best = deterministic_polish(best, data, cap) 421     q1 = json.loads(q1_path.read_text(encoding="utf-8"))
343     distances, times = validate_case(case_name, best, data, q1_case, 422     q2 = json.loads(q2_path.read_text(encoding="utf-8"))
    ↪ epsilon_h) 423     book = pd.ExcelFile(input_path)
344     delta1 = float(q1_case["Tmax"]) - float(q1_case["Tmin"]) 424     assert set(q1) == set(q2) == set(book.sheet_names)
345     delta2 = max(times) - min(times) 425     for case_name in book.sheet_names:
346     route_records = [] 426     data = load_case(book, case_name)
347     for k, (route, distance, work_h) in enumerate(zip(best, 427     routes = [route["sequence"][1:-1] for route in
    ↪ distances, times), start=1): 428     ↪ q2[case_name]["routes"]]
348     route_records.append( 429     distances, times = validate_case(case_name, routes, data,
349     { 430     ↪ q1[case_name], q2[case_name]["epsilon_h"]])
350     "uav": k, 431     assert abs(max(times) - q2[case_name]["Tmax"]) < 1.0e-8
351     "sequence": [0] + route + [0], 432     assert abs(min(times) - q2[case_name]["Tmin"]) < 1.0e-8
    433     for saved, distance, work_h in zip(q2[case_name]["routes"],
    ↪ distances, times):

```

```

414         assert abs(saved["distance_km"] - distance) < 1.0e-8
415         assert abs(saved["time_h"] - work_h) < 1.0e-8
416         assert saved["waiting_time_h"] == 0.0
417
418
419 def main() -> None:
420     parser = argparse.ArgumentParser()
421     parser.add_argument("--input", type=Path, required=True)
422     parser.add_argument("--q1-solution", type=Path, required=True)
423     parser.add_argument("--output-dir", type=Path, required=True)
424     parser.add_argument("--epsilon", type=float, default=0.0)
425     parser.add_argument("--seeds", type=int, default=8)
426     parser.add_argument("--iterations", type=int, default=30000)
427     parser.add_argument("--time-limit-per-case", type=float,
428         ↪ default=45.0)
429     parser.add_argument("--validate-only", action="store_true")
430     args = parser.parse_args()
431
432     output_json = args.output_dir / "q2_solution.json"
433     if args.validate_only:
434         validate_saved(args.input, args.q1_solution, output_json)
435         print(" 问题二保存结果独立校验通过。")
436         return
437
438     if args.epsilon < 0:
439         raise ValueError("epsilon 必须非负")
440     q1 = json.loads(args.q1_solution.read_text(encoding="utf-8"))
441     book = pd.ExcelFile(args.input)
442     results = {}
443     for case_name in book.sheet_names:
444         started = time.monotonic()
445         data = load_case(book, case_name)
446         results[case_name] = solve_case(
447             case_name,
448             data,
449             q1[case_name],
450             args.epsilon,
451             args.seeds,
452             args.iterations,
453             args.time_limit_per_case,
454         )
455         r = results[case_name]
456         print(
457             f"{case_name}: N={r['N']} Tmax={r['Tmax']:.6f} "
458             f"Tmin={r['Tmin']:.6f} delta={r['delta']:.6f} "
459             f"elapsed={time.monotonic()-started:.1f}s",
460             flush=True,
461         )
462     args.output_dir.mkdir(parents=True, exist_ok=True)
463     output_json.write_text(json.dumps(results, ensure_ascii=False,
464         ↪ indent=2), encoding="utf-8")
465     write_csv_outputs(results, args.output_dir)
466     validate_saved(args.input, args.q1_solution, output_json)
467     print(f" 问题二全部校验通过: {output_json}")
468
469 if __name__ == "__main__":
470     main()

```

附录 D 问题三源程序

以下收入算法 A、算法 B 及两套独立验证程序。算法 B 直接调用算法 A 文件中的数据结构和基础时空冲突函数，因此二者共同构成完整可运行程序。绘图、结果筛选和工作簿格式转换脚本仅放入电子支撑材料。

问题三算法 A 主程序

路径: submission_package/question3/program/q3_solver.py

```

1 """ 问题三: 动态圆形飞行管制下的多无人机协同巡检优化。
2
3 固定问题一/二采用的无人机数量, 以问题二路线为初始解; 时间以 8:00 为
4 零点、分钟为单位。目标按 (Tmax, delta, total_work) 词典序优化。等待仅在
5 航段或下一巡检服务受管制时发生, 不插入用于人为改善均衡性的空等待。
6 """
7
8 from __future__ import annotations
9
10 import argparse
11 import csv
12 import json
13 import math
14 import random
15 import time
16 from collections import Counter
17 from dataclasses import dataclass
18 from datetime import datetime, timedelta
19 from pathlib import Path
20
21 import pandas as pd
22
23
24 SPEED_KMH = 55.0
25 UNIT_KM = 0.1
26 SERVICE_MIN = 5.0
27 REQ = {"I": 3, "II": 2, "III": 1}
28 TOL = 1.0e-9
29
30
31 @dataclass(frozen=True)
32 class Zone:
33     zone_id: str
34     center: tuple[float, float]
35     radius: float
36     start_min: float
37     end_min: float
38
39     @property
40     def active(self) -> bool:
41         return self.end_min > self.start_min + TOL
42
43
44 @dataclass(frozen=True)
45 class CaseData:
46     coordinates: dict[int, tuple[float, float]]
47     expected_visits: dict[int, int]
48     zones: tuple[Zone, ...]
49
50
51 def clock_to_min(value: object) -> float:
52     text = str(value).strip()
53     parsed = datetime.strptime(text, "%H:%M")
54     return (parsed.hour - 8) * 60 + parsed.minute
55
56
57 def min_to_clock(value: float) -> str:
58     stamp = datetime(2000, 1, 1, 8, 0) + timedelta(minutes=value)
59     return stamp.strftime("%H:%M:%S")
60
61
62 def load_cases(points_path: Path, zones_path: Path) -> dict[str,
    63     point_book = pd.ExcelFile(points_path)
    64     zone_book = pd.ExcelFile(zones_path)
    65     assert point_book.sheet_names == zone_book.sheet_names
    66     result: dict[str, CaseData] = {}
    67     for case_name in point_book.sheet_names:
    68         points = pd.read_excel(point_book, sheet_name=case_name)
    69         zones_df = pd.read_excel(zone_book, sheet_name=case_name)
    70         coordinates = {0: (0.0, 0.0)}
    71         expected: dict[int, int] = {}
    72         for _, row in points.iterrows():
    73             point_id = int(row.Point_ID)
    74             coordinates[point_id] = (float(row.X_Coordinate),
    75                                     float(row.Y_Coordinate))
    76             expected[point_id] = REQ[str(row.Inspection_Level)]
    77         zones = tuple(
    78             Zone(
    79                 str(row.Zone_ID),
    80                 (float(row.Center_X), float(row.Center_Y)),
    81                 float(row.Radius),
    82                 clock_to_min(row.Start_Time),
    83                 clock_to_min(row.End_Time),
    84             )
    85             for _, row in zones_df.iterrows()
    86         )
    87         result[case_name] = CaseData(coordinates, expected, zones)
    88     return result
    89
    90 def point_inside(point: tuple[float, float], zone: Zone) -> bool:
    91     return math.dist(point, zone.center) <= zone.radius + 1.0e-10
    92
    93
    94 def segment_circle_interval(
    95     a: tuple[float, float], b: tuple[float, float], zone: Zone
    96 ) -> tuple[float, float] | None:
    97     """ 返回线段位于闭圆内的参数区间, 线段参数 lambda 属于 [0,1]。"""
    98     dx, dy = b[0] - a[0], b[1] - a[1]
    99     fx, fy = a[0] - zone.center[0], a[1] - zone.center[1]
    100     aa = dx * dx + dy * dy
    101     if aa <= TOL:
    102         return (0.0, 1.0) if point_inside(a, zone) else None
    103     bb = 2.0 * (fx * dx + fy * dy)
    104     cc = fx * fx + fy * fy - zone.radius * zone.radius
    105     disc = bb * bb - 4.0 * aa * cc
    106     if disc < -1.0e-10:
    107         return None
    108     disc = max(0.0, disc)
    109     root = math.sqrt(disc)
    110     lo = max(0.0, (-bb - root) / (2.0 * aa))
    111     hi = min(1.0, (-bb + root) / (2.0 * aa))
    112     if lo > hi + 1.0e-10:
    113         return None
    114     return lo, hi
    115
    116
    117 def overlaps(a0: float, a1: float, b0: float, b1: float) -> bool:
    118     return max(a0, b0) < min(a1, b1) - 1.0e-9
    119
    120
    121 def departure_blocks(
    122     a: tuple[float, float],
    123     b: tuple[float, float],
    124     travel_min: float,
    125     service_at_b: bool,
    126     zones: tuple[Zone, ...],
    127 ) -> list[tuple[float, float, str, str]]:
    128     blocks: list[tuple[float, float, str, str]] = []
    129     for zone in zones:
    130         if not zone.active:
    131             continue

```

```

132 interval = segment_circle_interval(a, b, zone)
133 if interval is not None:
134     lo, hi = interval
135     blocks.append(
136         (
137             zone.start_min - hi * travel_min,
138             zone.end_min - lo * travel_min,
139             zone.zone_id,
140             "flight",
141         )
142     )
143 if service_at_b and point_inside(b, zone):
144     blocks.append(
145         (
146             zone.start_min - travel_min - SERVICE_MIN,
147             zone.end_min - travel_min,
148             zone.zone_id,
149             "service",
150         )
151     )
152 return blocks
153
154 def unsafe_handoff_blocks(
155     current: tuple[float, float],
156     current_travel_min: float,
157     outgoing: list[tuple[float, float, str, str]],
158     zones: tuple[Zone, ...],
159 ) -> list[tuple[float, float, str, str]]:
160     """把“到点服务后不能在管制圆内等待”的约束前移到上一航段出发时刻。”
161     holding_zones = [zone for zone in zones if zone.active and
162         ↪ point_inside(current, zone)]
163     if not holding_zones:
164         return []
165     offset = current_travel_min + SERVICE_MIN
166     result: list[tuple[float, float, str, str]] = []
167     for block_start, block_end, cause_id, _ in outgoing:
168         for holding in holding_zones:
169             # 若完成服务时刻 r 落入出站禁用区间, 等待到 block_end;
170             # 只要 [r, block_end) 与当前点所在管制区重叠,
171             ↪ 就必须在上一段延迟。
172             unsafe_end = min(block_end, holding.end_min)
173             if block_end > holding.start_min + TOL and block_start < 246
174             ↪ unsafe_end - TOL:
175                 result.append(
176                     (
177                         block_start - offset,
178                         unsafe_end - offset,
179                         f"{holding.zone_id}/{cause_id}",
180                         "handoff",
181                     )
182                 )
183     return result
184
185 def earliest_departure(
186     ready: float, blocks: list[tuple[float, float, str, str]]
187 ) -> tuple[float, list[str]]:
188     depart = ready
189     reasons: set[str] = set()
190     for _ in range(len(blocks) + 2):
191         covering = [item for item in blocks if item[0] - TOL <=
192             ↪ depart < item[1] - TOL]
193         if not covering:
194             return depart, sorted(reasons)
195         reasons.update(f"{zone_id}:{kind}" for _, _, zone_id, kind in
196             ↪ covering)
197         depart = max(item[1] for item in covering)
198         raise RuntimeError(" 禁用出发区间合并未收敛")
199
200 def wait_is_safe(
201     node: int, point: tuple[float, float], start: float, end: float,
202     ↪ zones: tuple[Zone, ...]
203 ) -> bool:
204     if node == 0 or end <= start + TOL:
205         return True
206     for zone in zones:
207         if zone.active and point_inside(point, zone) and
208             ↪ overlaps(start, end, zone.start_min, zone.end_min):
209             return False
210     return True
211
212 def schedule_route(
213     tasks: list[int], data: CaseData, debug: bool = False,
214     ↪ release_min: float = 0.0
215 ) -> dict | None:
216     if not tasks or any(a == b for a, b in zip(tasks, tasks[1:])):
217         return None
218     sequence = [0] + tasks + [0]
219     leg_data = []
220     effective_blocks: list[list[tuple[float, float, str, str]]] = []
221     for a_id, b_id in zip(sequence, sequence[1:]):
222         a, b = data.coordinates[a_id], data.coordinates[b_id]
223         distance = math.dist(a, b) * UNIT_KM
224         travel = distance / SPEED_KMH * 60.0
225         leg_data.append((a_id, b_id, a, b, distance, travel))
226         effective_blocks.append(departure_blocks(a, b, travel, b_id
227             ↪ != 0, data.zones))
228     # 从路线末端向前传递: 若下一航段会迫使无人机在管制圆内等待,
229     # 则把相应禁用时段推回到上一安全节点的出发决策。
230     for idx in range(len(leg_data) - 2, -1, -1):
231         _, b_id, _, b, _, travel = leg_data[idx]
232         if b_id != 0:
233             effective_blocks[idx].extend(
234                 unsafe_handoff_blocks(b, travel, effective_blocks[idx
235                     ↪ + 1], data.zones)
236             )
237     now = release_min
238     distance_km = 0.0
239     wait_min = release_min
240     events: list[dict] = []
241     if release_min > TOL:
242         events.append(
243             {
244                 "type": "wait",
245                 "node": 0,
246                 "start_min": 0.0,
247                 "end_min": release_min,
248                 "duration_min": release_min,
249                 "reason_zone_ids": ["safe_release_after_all_zones"],
250             }
251         )
252     for leg, (a_id, b_id, a, b, distance, travel) in
253     ↪ enumerate(leg_data, start=1):
254         blocks = effective_blocks[leg - 1]
255         depart, reasons = earliest_departure(now, blocks)
256         if not wait_is_safe(a_id, a, now, depart, data.zones):
257             if debug:
258                 print(
259                     f"unsafe wait: node={a_id}, ready={now:.3f},
260                     ↪ depart={depart:.3f}, "
261                     f"next={b_id}, reasons={reasons}",
262                     flush=True,
263                 )
264             return None
265         if depart > now + TOL:
266             events.append(
267                 {
268                     "type": "wait",
269                     "node": a_id,
270                     "start_min": now,
271                     "end_min": depart,
272                     "duration_min": depart - now,
273                     "reason_zone_ids": reasons,
274                 }
275             )
276         wait_min += depart - now
277         arrival = depart + travel
278         events.append(
279             {
280                 "type": "flight",
281                 "leg": leg,
282                 "from": a_id,
283                 "to": b_id,
284                 "start_min": depart,
285                 "end_min": arrival,
286                 "duration_min": travel,
287                 "distance_km": distance,
288             }
289         )

```



```

283     distance_km += distance
284     now = arrival
285     if b_id != 0:
286         events.append(
287             {
288                 "type": "service",
289                 "node": b_id,
290                 "start_min": now,
291                 "end_min": now + SERVICE_MIN,
292                 "duration_min": SERVICE_MIN,
293             }
294         )
295     now += SERVICE_MIN
296     return {
297         "sequence": sequence,
298         "distance_km": distance_km,
299         "service_count": len(tasks),
300         "flight_time_h": distance_km / SPEED_KMH,
301         "wait_time_h": wait_min / 60.0,
302         "time_h": now / 60.0,
303         "return_clock": min_to_clock(now),
304         "events": events,
305     }
306
307 def valid_tasks(tasks: list[int]) -> bool:
308     return bool(tasks) and all(a != b for a, b in zip(tasks,
309         ↪ tasks[1:]))
310
311 class Evaluator:
312     def __init__(self, data: CaseData):
313         self.data = data
314         self.cache: dict[tuple[int, ...], dict | None] = {}
315
316     def route(self, tasks: list[int]) -> dict | None:
317         key = tuple(tasks)
318         if key not in self.cache:
319             result = schedule_route(tasks, self.data)
320             if result is None:
321                 safe_release = max((zone.end_min for zone in
322                     ↪ self.data.zones if zone.active), default=0.0)
323                 result = schedule_route(tasks, self.data,
324                     ↪ release_min=safe_release)
325             self.cache[key] = result
326         return self.cache[key]
327
328     def solution(self, routes: list[list[int]]) -> tuple[tuple[float,
329         ↪ float, float], list[dict]] | None:
330         schedules = [self.route(route) for route in routes]
331         if any(item is None for item in schedules):
332             return None
333         valid = [item for item in schedules if item is not None]
334         times = [item["time_h"] for item in valid]
335         key = (max(times), max(times) - min(times), sum(times))
336         return key, valid
337
338     def lex_better(a: tuple[float, ...], b: tuple[float, ...]) -> bool:
339         for x, y in zip(a, b):
340             if x < y - TOL:
341                 return True
342             if x > y + TOL:
343                 return False
344         return False
345
346     def propose(routes: list[list[int]], times: list[float], rng:
347         ↪ random.Random) -> list[list[int]] | None:
348         candidate = [route[:] for route in routes]
349         move = rng.random()
350         order = sorted(range(len(routes)), key=times.__getitem__)
351         if move < 0.48:
352             src = rng.choice(order[-min(2, len(order)):]) if rng.random()
353             ↪ < 0.82 else rng.randrange(len(routes))
354             choices = [idx for idx in range(len(routes)) if idx != src]
355             dst = rng.choice(order[: min(2, len(order))]) if rng.random()
356             ↪ < 0.82 else rng.choice(choices)
357             if dst == src:
358                 dst = rng.choice(choices)
359             if len(candidate[src]) <= 1:
360                 return None
361             pos = rng.randrange(len(candidate[src]))
362             node = candidate[src].pop(pos)
363             legal = [
364                 q for q in range(len(candidate[dst]) + 1)
365                 if (q == 0 or candidate[dst][q - 1] != node)
366                 and (q == len(candidate[dst]) or candidate[dst][q] !=
367                     ↪ node)
368             ]
369             if not legal:
370                 return None
371             candidate[dst].insert(rng.choice(legal), node)
372             elif move < 0.73:
373                 a, b = rng.sample(range(len(routes)), 2)
374                 ia, ib = rng.randrange(len(candidate[a])),
375                 ↪ rng.randrange(len(candidate[b]))
376                 candidate[a][ia], candidate[b][ib] = candidate[b][ib],
377                 ↪ candidate[a][ia]
378             else:
379                 ridx = rng.randrange(len(routes))
380                 route = candidate[ridx]
381                 if len(route) < 4:
382                     return None
383                 i, j = sorted(rng.sample(range(len(route)), 2))
384                 if i == j:
385                     return None
386                 route[i : j + 1] = reversed(route[i : j + 1])
387             if not all(valid_tasks(route) for route in candidate):
388                 return None
389             return candidate
390
391     def search_seed(
392         initial: list[list[int]],
393         evaluator: Evaluator,
394         seed: int,
395         iterations: int,
396         deadline: float,
397     ) -> tuple[list[list[int]], tuple[float, float, float], dict]:
398         rng = random.Random(seed)
399         current = [route[:] for route in initial]
400         evaluated = evaluator.solution(current)
401         if evaluated is None:
402             raise RuntimeError(" 问题二初始路线在问题三调度规则下不可行")
403         current_key, current_schedules = evaluated
404         best = [route[:] for route in current]
405         best_key = current_key
406         attempted = accepted = 0
407
408         for iteration in range(iterations):
409             if time.monotonic() >= deadline:
410                 break
411             times = [item["time_h"] for item in current_schedules]
412             candidate = propose(current, times, rng)
413             if candidate is None:
414                 continue
415             attempted += 1
416             evaluated = evaluator.solution(candidate)
417             if evaluated is None:
418                 continue
419             key, schedules = evaluated
420             accept = lex_better(key, current_key)
421             if not accept:
422                 progress = iteration / max(1, iterations - 1)
423                 temperature = 0.12 * (1.0 - progress) + 0.002
424                 current_score = current_key[0] + 0.18 * current_key[1] +
425                 ↪ 0.002 * current_key[2]
426                 candidate_score = key[0] + 0.18 * key[1] + 0.002 * key[2]
427                 loss = max(0.0, candidate_score - current_score)
428                 accept = rng.random() < math.exp(-loss / temperature)
429             if accept:
430                 current, current_key, current_schedules = candidate, key,
431                 ↪ schedules
432                 accepted += 1
433                 if lex_better(key, best_key):
434                     best, best_key = [route[:] for route in candidate],
435                     ↪ key
436         return best, best_key, {
437             "seed": seed,

```

```

433     "attempted_moves": attempted,
434     "accepted_moves": accepted,
435     "Tmax_h": best_key[0],
436     "delta_h": best_key[1],
437     "total_work_h": best_key[2],
438 }
439
440
441 def validate_counts(routes: list[list[int]], data: CaseData) -> None:
442     found = Counter(node for route in routes for node in route)
443     assert dict(found) == data.expected_visits
444     assert all(valid_tasks(route) for route in routes)
445
446
447 def zone_summary(data: CaseData) -> dict:
448     active = [zone for zone in data.zones if zone.active]
449     involved = {
450         node
451         for node, point in data.coordinates.items()
452         if node != 0 and any(point_inside(point, zone) for zone in
453                               ↪ active)
454     }
455     return {
456         "zone_count": len(data.zones),
457         "positive_duration_count": len(active),
458         "zero_duration_count": len(data.zones) - len(active),
459         "involved_point_count": len(involved),
460         "involved_points": sorted(involved),
461     }
462
463 def solve_case(
464     case_name: str,
465     data: CaseData,
466     initial: list[list[int]],
467     seeds: int,
468     iterations: int,
469     seconds: float,
470 ) -> dict:
471     validate_counts(initial, data)
472     evaluator = Evaluator(data)
473     baseline = evaluator.solution(initial)
474     if baseline is None:
475         for idx, route in enumerate(initial, start=1):
476             if evaluator.route(route) is None:
477                 print(f"{case_name} UAV{idx} diagnostic", flush=True)
478                 schedule_route(route, data, debug=True)
479                 raise RuntimeError(f"{case_name}:
480                               ↪ 初始路线无法生成合法动态调度")
481     best_routes = [route[:] for route in initial]
482     best_key, _ = baseline
483     history = []
484     deadline = time.monotonic() + seconds
485     for offset in range(seeds):
486         routes, key, report = search_seed(
487             initial,
488             evaluator,
489             20260818 + offset,
490             iterations,
491             deadline,
492         )
493         history.append(report)
494         if lex_better(key, best_key):
495             best_routes, best_key = routes, key
496         if time.monotonic() >= deadline:
497             break
498     validate_counts(best_routes, data)
499     final = evaluator.solution(best_routes)
500     assert final is not None
501     key, schedules = final
502     routes_output = []
503     for uav, schedule in enumerate(schedules, start=1):
504         routes_output.append({"uav": uav, **schedule})
505     return {
506         "N": len(best_routes),
507         "Tmax": key[0],
508         "Tmin": min(item["time_h"] for item in schedules),
509         "delta": key[1],
510         "total_work_h": key[2],
511         "total_wait_min": sum(item["wait_time_h"] * 60 for item in
512                               ↪ schedules),
513         "latest_return": min_to_clock(key[0] * 60),
514         "time_origin": "08:00",
515         "deadline_17_00": False,
516         "objective_order": ["Tmax", "delta", "total_work_h"],
517         "zone_summary": zone_summary(data),
518         "routes": routes_output,
519         "search_history": history,
520     }
521
522 def write_csv(results: dict, output_dir: Path) -> None:
523     output_dir.mkdir(parents=True, exist_ok=True)
524     with (output_dir / "summary.csv").open("w", newline="",
525       ↪ encoding="utf-8-sig") as handle:
526         writer = csv.writer(handle)
527         writer.writerow(["Case", "N", "Tmax_h", "Tmin_h", "delta_h",
528           ↪ "wait_min", "latest_return"])
529         for case_name, case in results.items():
530             writer.writerow([case_name, case["N"], case["Tmax"],
531               ↪ case["Tmin"], case["delta"], case["total_wait_min"],
532               ↪ case["latest_return"]])
533         for case_name, case in results.items():
534             with (output_dir / f"{case_name}_routes.csv").open("w",
535               ↪ newline="", encoding="utf-8-sig") as handle:
536                 writer = csv.writer(handle)
537                 writer.writerow(["UAV", "Sequence", "Distance_km",
538                   ↪ "Service_Count", "Wait_min", "Work_h", "Return"])
539                 for route in case["routes"]:
540                     writer.writerow([route["uav"], "-".join(map(str,
541                       ↪ route["sequence"])), route["distance_km"],
542                       ↪ route["service_count"], route["wait_time_h"] *
543                       ↪ 60, route["time_h"], route["return_clock"]])
544
545 def main() -> None:
546     parser = argparse.ArgumentParser()
547     parser.add_argument("--points", type=Path, required=True)
548     parser.add_argument("--zones", type=Path, required=True)
549     parser.add_argument("--q2-solution", type=Path, required=True)
550     parser.add_argument("--output-dir", type=Path, required=True)
551     parser.add_argument("--seeds", type=int, default=5)
552     parser.add_argument("--iterations", type=int, default=25000)
553     parser.add_argument("--seconds-per-case", type=float,
554       ↪ default=35.0)
555     args = parser.parse_args()
556
557     cases = load_cases(args.points, args.zones)
558     q2 = json.loads(args.q2_solution.read_text(encoding="utf-8"))
559     assert set(cases) == set(q2)
560     results = {}
561     for case_name, data in cases.items():
562         initial = [[int(x) for x in route["sequence"][1:-1]] for
563           ↪ route in q2[case_name]["routes"]]
564         started = time.monotonic()
565         results[case_name] = solve_case(
566             case_name,
567             data,
568             initial,
569             args.seeds,
570             args.iterations,
571             args.seconds_per_case,
572         )
573         elapsed = time.monotonic() - started
574         case = results[case_name]
575         print(
576             f"{case_name}: N={case['N']}, Tmax={case['Tmax']:.6f} h,
577             ↪ "
578             f"Tmin={case['Tmin']:.6f} h, delta={case['delta']:.6f} h,
579             ↪ "
580             f"wait={case['total_wait_min']:.3f} min,
581             ↪ elapsed={elapsed:.1f}s",
582             flush=True,
583         )
584     args.output_dir.mkdir(parents=True, exist_ok=True)
585     output = args.output_dir / "q3_solution.json"
586     output.write_text(json.dumps(results, ensure_ascii=False,
587       ↪ indent=2, encoding="utf-8"))
588     write_csv(results, args.output_dir)
589     print(output)

```

```

578
579 if __name__ == "__main__":
580     main()

```

问题三算法 B 主程序

路径: submission_package/question3/program/q3_solver_enhanced.py

```

1 """ 问题三算法 B: 时变圆形禁飞区下的分层增强求解器。
2
3 与基准算法的区别:
4 1. 每次巡检使用唯一任务副本编号;
5 2. 对受阻转场比较“等待后直飞”和“边界离散可见图绕飞”;
6 3. 阶段 A 只搜索最小 Tmax, 阶段 B 在给定 epsilon 容差内搜索最小 delta;
7 4. 固定问题二机队规模是正式方案, 额外无人机仅作为敏感性分析。
8
9 圆边界用外扩采样环及直线弦近似。采样环半径除以 cos(pi/m), 保证相邻弦
10 不进入原禁飞圆内部; 所有最终飞行事件仍由解析线段—圆—时间区间复核。
11 """
12
13 from __future__ import annotations
14
15 import argparse
16 import csv
17 import heapq
18 import json
19 import math
20 import random
21 import time
22 from collections import Counter, defaultdict
23 from dataclasses import dataclass
24 from pathlib import Path
25
26 import q3_solver as baseline
27
28 TOL = 1.0e-9
29
30
31
32 @dataclass(frozen=True)
33 class GeoNode:
34     label: str
35     xy: tuple[float, float]
36
37
38 def task_point(task_id: str) -> int:
39     return int(task_id.split("#", 1)[0][1:])
40
41
42 def build_unique_copies(initial_physical: list[list[int]]) ->
    tuple[list[list[str]], dict[str, int]]:
43     counters: defaultdict[int, int] = defaultdict(int)
44     mapping: dict[str, int] = {}
45     routes: list[list[str]] = []
46     for route in initial_physical:
47         copies: list[str] = []
48         for point in route:
49             counters[point] += 1
50             task_id = f"P{point:03d}#{counters[point]}"
51             mapping[task_id] = point
52             copies.append(task_id)
53         routes.append(copies)
54     return routes, mapping
55
56
57 def expected_copy_ids(data: baseline.CaseData) -> set[str]:
58     return {
59         f"P{point:03d}#{copy_no}"
60         for point, count in data.expected_visits.items()
61         for copy_no in range(1, count + 1)
62     }
63
64
65 def valid_copy_route(route: list[str], mapping: dict[str, int]) ->
    bool:

```

```

66     if not route:
67         return False
68     physical = [mapping[item] for item in route]
69     return all(a != b for a, b in zip(physical, physical[1:]))
70
71
72 def validate_copy_routes(
73     routes: list[list[str]], data: baseline.CaseData, mapping:
    dict[str, int]
74 ) -> None:
75     flat = [item for route in routes for item in route]
76     assert len(flat) == len(set(flat)), "任务副本编号重复"
77     assert set(flat) == expected_copy_ids(data), "任务副本缺失或多余"
78     assert set(mapping) == expected_copy_ids(data)
79     assert all(valid_copy_route(route, mapping) for route in routes)
80
81
82 def point_segment_distance(
83     p: tuple[float, float], a: tuple[float, float], b: tuple[float,
    float]
84 ) -> float:
85     dx, dy = b[0] - a[0], b[1] - a[1]
86     denom = dx * dx + dy * dy
87     if denom <= TOL:
88         return math.dist(p, a)
89     lam = ((p[0] - a[0]) * dx + (p[1] - a[1]) * dy) / denom
90     lam = min(1.0, max(0.0, lam))
91     q = (a[0] + lam * dx, a[1] + lam * dy)
92     return math.dist(p, q)
93
94
95 class DetourRouter:
96     def __init__(
97         self,
98         data: baseline.CaseData,
99         ring_samples: int = 16,
100         safety_margin: float = 0.0,
101     ) -> None:
102         if ring_samples < 8:
103             raise ValueError("ring_samples 至少为 8")
104         self.data = data
105         self.ring_samples = ring_samples
106         self.safety_margin = safety_margin
107         self.path_cache: dict[tuple[int, int, tuple[int, ...]],
    list[GeoNode] | None] = {}
108
109     def _zone_radius(self, zone: baseline.Zone) -> float:
110         return zone.radius + self.safety_margin
111
112     def _static_clear(
113         self,
114         a: tuple[float, float],
115         b: tuple[float, float],
116         obstacle_indices: tuple[int, ...],
117     ) -> bool:
118         for index in obstacle_indices:
119             zone = self.data.zones[index]
120             radius = self._zone_radius(zone)
121             if point_segment_distance(zone.center, a, b) < radius -
    1.0e-8:
122                 return False
123         return True
124
125     def _ring_nodes(self, obstacle_indices: tuple[int, ...]) ->
    list[GeoNode]:
126         nodes: list[GeoNode] = []
127         inflation = 1.0 / math.cos(math.pi / self.ring_samples)
128         for index in obstacle_indices:
129             zone = self.data.zones[index]
130             radius = self._zone_radius(zone) * inflation + 1.0e-6
131             for sample in range(self.ring_samples):
132                 angle = 2.0 * math.pi * sample / self.ring_samples
133                 xy = (
134                     zone.center[0] + radius * math.cos(angle),
135                     zone.center[1] + radius * math.sin(angle),
136                 )
137                 # 位于其他同时考虑的圆内部的采样点不能作为安全转向点。
138                 if any(
139                     other != index
140                     and math.dist(xy, self.data.zones[other].center)
141                     < self._zone_radius(self.data.zones[other]) -
    1.0e-8

```

```

142         for other in obstacle_indices
143     ):
144         continue
145     nodes.append(GeoNode(f"P{index + 1}:B{sample}", xy))
146     return nodes
147
148 def _static_detour(
149     self,
150     a_id: int,
151     b_id: int,
152     obstacles: tuple[int, ...],
153 ) -> list[GeoNode] | None:
154     key = (a_id, b_id, obstacles)
155     if key in self.path_cache:
156         return self.path_cache[key]
157     source = GeoNode(f"P{a_id}", self.data.coordinates[a_id])
158     target = GeoNode(f"P{b_id}", self.data.coordinates[b_id])
159     nodes = [source, target, *self._ring_nodes(obstacles)]
160     if any(
161         math.dist(source.xy, self.data.zones[index].center)
162         < self._zone_radius(self.data.zones[index]) - 1.0e-8
163         or math.dist(target.xy, self.data.zones[index].center)
164         < self._zone_radius(self.data.zones[index]) - 1.0e-8
165         for index in obstacles
166     ):
167         self.path_cache[key] = None
168         return None
169
170     adjacency: list[list[tuple[int, float]]] = [[] for _ in
171     ↪ nodes]
172     for i in range(len(nodes)):
173         for j in range(i + 1, len(nodes)):
174             if self._static_clear(nodes[i].xy, nodes[j].xy,
175             ↪ obstacles):
176                 distance = math.dist(nodes[i].xy, nodes[j].xy)
177                 adjacency[i].append((j, distance))
178                 adjacency[j].append((i, distance))
179
180     dist = [math.inf] * len(nodes)
181     previous = [-1] * len(nodes)
182     dist[0] = 0.0
183     heap: list[tuple[float, int]] = [(0.0, 0)]
184     while heap:
185         current, u = heapq.heappop(heap)
186         if current > dist[u] + TOL:
187             continue
188         if u == 1:
189             break
190         for v, weight in adjacency[u]:
191             candidate = current + weight
192             if candidate < dist[v] - TOL:
193                 dist[v] = candidate
194                 previous[v] = u
195                 heapq.heappush(heap, (candidate, v))
196     if not math.isfinite(dist[1]):
197         self.path_cache[key] = None
198         return None
199     order: list[int] = []
200     cursor = 1
201     while cursor >= 0:
202         order.append(cursor)
203         if cursor == 0:
204             break
205         cursor = previous[cursor]
206     path = [nodes[index] for index in reversed(order)]
207     self.path_cache[key] = path
208     return path
209
210 def _simulate_polyline(
211     self,
212     nodes: list[GeoNode],
213     ready: float,
214     service_at_target: bool,
215     source_is_base: bool,
216     mode: str,
217 ) -> dict | None:
218     now = ready
219     total_distance = 0.0
220     events: list[dict] = []
221     for index, (left, right) in enumerate(zip(nodes, nodes[1:])):
222         distance_km = math.dist(left.xy, right.xy) *
223         ↪ baseline.UNIT_KM
224         travel_min = distance_km / baseline.SPEED_KMH * 60.0
225         last = index == len(nodes) - 2
226         blocks = baseline.departure_blocks(
227             left.xy,
228             right.xy,
229             travel_min,
230             service_at_target and last,
231             self.data.zones,
232         )
233         depart, reasons = baseline.earliest_departure(now,
234         ↪ blocks)
235         allow_base_wait = source_is_base and index == 0
236         node_number = 0 if allow_base_wait else -1
237         if not baseline.wait_is_safe(node_number, left.xy, now,
238         ↪ depart, self.data.zones):
239             return None
240         if depart > now + TOL:
241             events.append(
242                 {
243                     "type": "wait",
244                     "node_label": left.label,
245                     "node_xy": list(left.xy),
246                     "start_min": now,
247                     "end_min": depart,
248                     "duration_min": depart - now,
249                     "reason_zone_ids": reasons,
250                     "mode": mode,
251                 }
252             )
253         arrival = depart + travel_min
254         events.append(
255             {
256                 "type": "flight",
257                 "from_label": left.label,
258                 "to_label": right.label,
259                 "from_xy": list(left.xy),
260                 "to_xy": list(right.xy),
261                 "start_min": depart,
262                 "end_min": arrival,
263                 "duration_min": travel_min,
264                 "distance_km": distance_km,
265                 "mode": mode,
266             }
267         )
268         total_distance += distance_km
269         now = arrival
270     return {
271         "arrival_min": now,
272         "distance_km": total_distance,
273         "events": events,
274         "mode": mode,
275     }
276
277 def travel(
278     self,
279     a_id: int,
280     b_id: int,
281     ready: float,
282     service_at_target: bool,
283 ) -> dict | None:
284     direct_nodes = [
285         GeoNode(f"P{a_id}", self.data.coordinates[a_id]),
286         GeoNode(f"P{b_id}", self.data.coordinates[b_id]),
287     ]
288     direct = self._simulate_polyline(
289         direct_nodes,
290         ready,
291         service_at_target,
292         a_id == 0,
293         "direct_or_wait",
294     )
295     if direct is not None and not any(event["type"] == "wait" for
296     ↪ event in direct["events"]):
297         return direct
298
299     horizon = direct["arrival_min"] if direct is not None else
300     ↪ max(
301         (zone.end_min for zone in self.data.zones if
302         ↪ zone.active), default=ready
303     )
304     obstacles = tuple(

```

```

297         index
298         for index, zone in enumerate(self.data.zones)
299         if zone.active
300         and baseline.overlaps(ready, max(ready + TOL, horizon),
301             ↪ zone.start_min, zone.end_min)
302     )
303     detour = None
304     if obstacles:
305         path = self._static_detour(a_id, b_id, obstacles)
306         if path is not None and len(path) > 2:
307             detour = self._simulate_polyline(
308                 path,
309                 ready,
310                 service_at_target,
311                 a_id == 0,
312                 "boundary_visibility_detour",
313             )
314         candidates = [item for item in (direct, detour) if item is
315             ↪ not None]
316         if not candidates:
317             return None
318         return min(candidates, key=lambda item: (item["arrival_min"],
319             ↪ item["distance_km"]))
320
321 def _schedule_copy_route_with_detours(
322     route: list[str],
323     mapping: dict[str, int],
324     data: baseline.CaseData,
325     router: DetourRouter,
326 ) -> dict | None:
327     if not valid_copy_route(route, mapping):
328         return None
329     now = 0.0
330     distance_km = 0.0
331     events: list[dict] = []
332     physical = [mapping[item] for item in route]
333     point_sequence = [0, *physical, 0]
334     task_sequence = ["BASE", *route, "BASE"]
335     for leg, (a_id, b_id) in enumerate(zip(point_sequence,
336         ↪ point_sequence[1:]), start=1):
337         movement = router.travel(a_id, b_id, now, b_id != 0)
338         if movement is None:
339             return None
340         for event in movement["events"]:
341             event = {"leg": leg, **event}
342             events.append(event)
343         now = movement["arrival_min"]
344         distance_km += movement["distance_km"]
345         if b_id != 0:
346             task_id = route[leg - 1]
347             events.append(
348                 {
349                     "type": "service",
350                     "task_id": task_id,
351                     "point_id": b_id,
352                     "node_xy": list(data.coordinates[b_id]),
353                     "start_min": now,
354                     "end_min": now + baseline.SERVICE_MIN,
355                     "duration_min": baseline.SERVICE_MIN,
356                 }
357             )
358         now += baseline.SERVICE_MIN
359     wait_min = sum(event["duration_min"] for event in events if
360         ↪ event["type"] == "wait")
361     detour_legs = len(
362         {
363             event["leg"]
364             for event in events
365             if event["type"] == "flight" and event.get("mode") ==
366             ↪ "boundary_visibility_detour"
367         }
368     )
369     return {
370         "sequence": point_sequence,
371         "task_sequence": task_sequence,
372         "distance_km": distance_km,
373         "service_count": len(route),
374         "flight_time_h": distance_km / baseline.SPEED_KMH,
375         "wait_time_h": wait_min / 60.0,
376         "time_h": now / 60.0,
377         "return_clock": baseline.min_to_clock(now),
378     }
379
380 "detour_leg_count": detour_legs,
381 "used_baseline_handoff_fallback": False,
382 "events": events,
383 }
384
385 def _baseline_handoff_fallback(
386     route: list[str], mapping: dict[str, int], data:
387     ↪ baseline.CaseData
388 ) -> dict | None:
389     """ 当逐航段绕飞评价陷入不安全驻留时, 使用基准算法的反向等待传播。"""
390     physical = [mapping[item] for item in route]
391     scheduled = baseline.schedule_route(physical, data)
392     if scheduled is None:
393         safe_release = max((zone.end_min for zone in data.zones if
394             ↪ zone.active), default=0.0)
395         scheduled = baseline.schedule_route(physical, data,
396             ↪ release_min=safe_release)
397     if scheduled is None:
398         return None
399     service_cursor = 0
400     converted: list[dict] = []
401     for event in scheduled["events"]:
402         item = dict(event)
403         if event["type"] == "flight":
404             a_id, b_id = int(event["from"]), int(event["to"])
405             item.update(
406                 {
407                     "from_label": f"P{a_id}",
408                     "to_label": f"P{b_id}",
409                     "from_xy": list(data.coordinates[a_id]),
410                     "to_xy": list(data.coordinates[b_id]),
411                     "mode": "direct_with_backward_handoff",
412                 }
413             )
414         elif event["type"] == "wait":
415             node = int(event["node"])
416             item.update(
417                 {
418                     "node_label": f"P{node}",
419                     "node_xy": list(data.coordinates[node]),
420                     "mode": "direct_with_backward_handoff",
421                 }
422             )
423         elif event["type"] == "service":
424             item.update(
425                 {
426                     "task_id": route[service_cursor],
427                     "point_id": int(event["node"]),
428                     "node_xy":
429                     ↪ list(data.coordinates[int(event["node"])]),
430                 }
431             )
432             service_cursor += 1
433             converted.append(item)
434     return {
435         **{key: value for key, value in scheduled.items() if key !=
436             ↪ "events"},
437         "task_sequence": ["BASE", *route, "BASE"],
438         "detour_leg_count": 0,
439         "used_baseline_handoff_fallback": True,
440         "events": converted,
441     }
442
443 def schedule_copy_route(
444     route: list[str],
445     mapping: dict[str, int],
446     data: baseline.CaseData,
447     router: DetourRouter,
448 ) -> dict | None:
449     enhanced = _schedule_copy_route_with_detours(route, mapping,
450         ↪ data, router)
451     if enhanced is not None:
452         return enhanced
453     return _baseline_handoff_fallback(route, mapping, data)
454
455 class EnhancedEvaluator:
456     def __init__(
457         self,

```

```

448     data: baseline.CaseData,
449     mapping: dict[str, int],
450     ring_samples: int,
451     safety_margin: float,
452 ) -> None:
453     self.data = data
454     self.mapping = mapping
455     self.router = DetourRouter(data, ring_samples, safety_margin)
456     self.cache: dict[tuple[str, ...], dict | None] = {}
457
458 def route(self, tasks: list[str]) -> dict | None:
459     key = tuple(tasks)
460     if key not in self.cache:
461         self.cache[key] = schedule_copy_route(tasks,
462             ↪ self.mapping, self.data, self.router)
463     return self.cache[key]
464
465 def solution(self, routes: list[list[str]]) -> tuple[tuple[float, float, float], list[dict]] | None:
466     ↪ float, float], list[dict]] | None:
467     schedules = [self.route(route) for route in routes]
468     if any(item is None for item in schedules):
469         return None
470     valid = [item for item in schedules if item is not None]
471     times = [item["time_h"] for item in valid]
472     return (max(times), max(times) - min(times), sum(times)),
473     ↪ valid
474
475 def propose(
476     routes: list[list[str]],
477     times: list[float],
478     mapping: dict[str, int],
479     rng: random.Random,
480 ) -> list[list[str]] | None:
481     candidate = [route[:] for route in routes]
482     order = sorted(range(len(routes)), key=times.__getitem__)
483     move = rng.random()
484     if move < 0.42:
485         src = rng.choice(order[-min(2, len(order)):]) if rng.random() < 0.8 else rng.randrange(len(routes))
486         choices = [idx for idx in range(len(routes)) if idx != src]
487         dst = rng.choice(order[: min(2, len(order))]) if rng.random() < 0.8 else rng.choice(choices)
488         ↪ < 0.8 else rng.choice(choices)
489         if len(candidate[src]) <= 1:
490             return None
491         task = candidate[src].pop(rng.randrange(len(candidate[src])))
492         point = mapping[task]
493         legal = [
494             pos
495             for pos in range(len(candidate[dst]) + 1)
496             if (pos == 0 or mapping[candidate[dst][pos - 1]] !=
497                 ↪ point)
498             and (pos == len(candidate[dst]) or
499                 ↪ mapping[candidate[dst][pos]] != point)
500         ]
501         if not legal:
502             return None
503         candidate[dst].insert(rng.choice(legal), task)
504     elif move < 0.68:
505         a, b = rng.sample(range(len(routes)), 2)
506         ia, ib = rng.randrange(len(candidate[a])),
507             ↪ rng.randrange(len(candidate[b]))
508         candidate[a][ia], candidate[b][ib] = candidate[b][ib],
509             ↪ candidate[a][ia]
510     elif move < 0.84:
511         ridx = rng.randrange(len(routes))
512         if len(candidate[ridx]) < 4:
513             return None
514         i, j = sorted(rng.sample(range(len(candidate[ridx])), 2))
515         candidate[ridx][i : j + 1] = reversed(candidate[ridx][i : j
516             ↪ 1])
517     else:
518         a, b = rng.sample(range(len(routes)), 2)
519         if len(candidate[a]) < 2 or len(candidate[b]) < 2:
520             return None
521         ia, ja = sorted(rng.sample(range(len(candidate[a]) + 1), 2))
522         ib, jb = sorted(rng.sample(range(len(candidate[b]) + 1), 2))
523         if ia == ja or ib == jb:
524             return None
525         seg_a, seg_b = candidate[a][ia:ja], candidate[b][ib:jb]
526         candidate[a][ia:ja], candidate[b][ib:jb] = seg_b, seg_a
527
528 if not all(valid_copy_route(route, mapping) for route in
529     ↪ candidate):
530     return None
531     return candidate
532
533 def stage_a_search(
534     initial: list[list[str]],
535     evaluator: EnhancedEvaluator,
536     mapping: dict[str, int],
537     rng: random.Random,
538     iterations: int,
539     deadline: float,
540 ) -> tuple[list[list[str]], tuple[float, float, float], dict]:
541     current = [route[:] for route in initial]
542     evaluated = evaluator.solution(current)
543     if evaluated is None:
544         raise RuntimeError(" 增强算法无法调度初始路线")
545     current_key, current_schedules = evaluated
546     best, best_key = [route[:] for route in current], current_key
547     attempted = accepted = 0
548     for iteration in range(iterations):
549         if time.monotonic() >= deadline:
550             break
551         candidate = propose(current, [item["time_h"] for item in
552             ↪ current_schedules], mapping, rng)
553         if candidate is None:
554             continue
555         attempted += 1
556         evaluated = evaluator.solution(candidate)
557         if evaluated is None:
558             continue
559         key, schedules = evaluated
560         progress = iteration / max(1, iterations - 1)
561         temperature = 0.10 * (1.0 - progress) + 0.001
562         loss = key[0] - current_key[0]
563         accept = loss < -TOL or rng.random() < math.exp(-max(0.0,
564             ↪ loss) / temperature)
565         if accept:
566             current, current_key, current_schedules = candidate, key,
567             ↪ schedules
568             accepted += 1
569             if key[0] < best_key[0] - TOL or (
570                 abs(key[0] - best_key[0]) <= TOL and key[2] <
571                 ↪ best_key[2] - TOL
572             ):
573                 best, best_key = [route[:] for route in candidate],
574                 ↪ key
575     return best, best_key, {
576         "attempted_moves": attempted,
577         "accepted_moves": accepted,
578         "Tmax_star_h": best_key[0],
579         "secondary_metrics_not_optimized": {"delta_h": best_key[1],
580             ↪ "total_work_h": best_key[2]},
581     }
582
583 def stage_b_search(
584     stage_a_routes: list[list[str]],
585     tmax_star: float,
586     epsilon: float,
587     evaluator: EnhancedEvaluator,
588     mapping: dict[str, int],
589     rng: random.Random,
590     iterations: int,
591     deadline: float,
592 ) -> tuple[list[list[str]], tuple[float, float, float], dict]:
593     threshold = (1.0 + epsilon) * tmax_star
594     current = [route[:] for route in stage_a_routes]
595     evaluated = evaluator.solution(current)
596     assert evaluated is not None
597     current_key, current_schedules = evaluated
598     best, best_key = [route[:] for route in current], current_key
599     attempted = accepted = rejected_by_threshold = 0
600     for iteration in range(iterations):
601         if time.monotonic() >= deadline:
602             break
603         candidate = propose(current, [item["time_h"] for item in
604             ↪ current_schedules], mapping, rng)
605         if candidate is None:
606             continue
607         attempted += 1

```

```

593     evaluated = evaluator.solution(candidate)
594     if evaluated is None:
595         continue
596     key, schedules = evaluated
597     if key[0] > threshold + TOL:
598         rejected_by_threshold += 1
599         continue
600     current_obj = (current_key[1], current_key[2],
601                  ↪ current_key[0])
602     candidate_obj = (key[1], key[2], key[0])
603     progress = iteration / max(1, iterations - 1)
604     temperature = 0.06 * (1.0 - progress) + 0.0005
605     loss = (candidate_obj[0] - current_obj[0]) + 0.002 * (
606         candidate_obj[1] - current_obj[1]
607     )
608     accept = candidate_obj < current_obj or rng.random() <
609     ↪ math.exp(-max(0.0, loss) / temperature)
610     if accept:
611         current, current_key, current_schedules = candidate, key,
612         ↪ schedules
613         accepted += 1
614         if (key[1], key[2], key[0]) < (best_key[1], best_key[2],
615         ↪ best_key[0]):
616             best, best_key = [route[:] for route in candidate],
617             ↪ key
618     return best, best_key, {
619         "epsilon": epsilon,
620         "Tmax_limit_h": threshold,
621         "attempted_moves": attempted,
622         "accepted_moves": accepted,
623         "rejected_by_Tmax_limit": rejected_by_threshold,
624         "delta_h": best_key[1],
625         "Tmax_h": best_key[0],
626         "total_work_h": best_key[2],
627     }
628
629 def add_uavs(initial: list[list[str]], extra: int) ->
630     ↪ list[list[str]]:
631     routes = [route[:] for route in initial]
632     for _ in range(extra):
633         src = max(range(len(routes)), key=lambda idx:
634         ↪ len(routes[idx]))
635         if len(routes[src]) <= 1:
636             raise ValueError(" 无法继续拆分非空路线")
637         midpoint = len(routes[src]) // 2
638         new_route = routes[src][midpoint:]
639         routes[src] = routes[src][:midpoint]
640         routes.append(new_route)
641     return routes
642
643 def solve_case_n(
644     case_name: str,
645     data: baseline.CaseData,
646     initial: list[list[str]],
647     mapping: dict[str, int],
648     epsilon: float,
649     ring_samples: int,
650     safety_margin: float,
651     stage_a_iterations: int,
652     stage_b_iterations: int,
653     seconds: float,
654     seed: int,
655 ) -> dict:
656     validate_copy_routes(initial, data, mapping)
657     evaluator = EnhancedEvaluator(data, mapping, ring_samples,
658     ↪ safety_margin)
659     started = time.monotonic()
660     phase_deadline = started + seconds / 2.0
661     rng = random.Random(seed)
662     stage_a_routes, stage_a_key, stage_a_report = stage_a_search(
663     initial, evaluator, mapping, rng, stage_a_iterations,
664     ↪ phase_deadline
665     )
666     original_stage_a_tmax = stage_a_key[0]
667     stage_b_routes, stage_b_key, stage_b_report = stage_b_search(
668     stage_a_routes,
669     stage_a_key[0],
670     epsilon,
671     evaluator,
672     mapping,
673     rng,
674     stage_b_iterations,
675     time.monotonic() + max(2.0, seconds / 3.0),
676     )
677     stage_a_report["initial_Tmax_star_h"] = original_stage_a_tmax
678     stage_a_report["Tmax_star_h"] = stage_a_key[0]
679     stage_a_report["refined_from_stage_b"] = bool(refinements)
680     stage_a_report["refinement_history"] = refinements
681     validate_copy_routes(stage_b_routes, data, mapping)
682     evaluated = evaluator.solution(stage_b_routes)
683     assert evaluated is not None
684     key, schedules = evaluated
685     assert key[0] <= (1.0 + epsilon) * stage_a_key[0] + TOL
686     return {
687         "case": case_name,
688         "N": len(stage_b_routes),
689         "Tmax": key[0],
690         "Tmin": min(item["time_h"] for item in schedules),
691         "delta": key[1],
692         "total_work_h": key[2],
693         "total_wait_min": sum(item["wait_time_h"] * 60.0 for item in
694         ↪ schedules),
695         "latest_return": baseline.min_to_clock(key[0] * 60.0),
696         "time_origin": "08:00",
697         "deadline_17_00": False,
698         "zone_policy": "half_open; zero-duration zones excluded",
699         "task_copy_policy": "unique task IDs Pxxx#copy_no",
700         "detour_model": {
701             "type": "sampled boundary visibility graph",
702             "ring_samples": ring_samples,
703             "safety_margin_coordinate_units": safety_margin,
704         },
705         "stage_a": stage_a_report,
706         "stage_b": stage_b_report,
707         "routes": [
708             {"uav": uav, **schedule}
709             for uav, schedule in enumerate(schedules, start=1)
710         ],
711     }
712
713 def write_summary(results: dict, output_dir: Path) -> None:
714     output_dir.mkdir(parents=True, exist_ok=True)
715     with (output_dir / "summary_enhanced.csv").open("w", newline="",
716     ↪ encoding="utf-8-sig") as handle:
717         writer = csv.writer(handle)
718         writer.writerow(
719             [
720                 "Case",
721                 "Scenario",
722                 "N",
723                 "StageA_TmaxStar_h",
724                 "Epsilon",
725                 "Final_Tmax_h",
726                 "Final_Tmin_h",
727                 "Final_delta_h",
728             ]
729         )

```

```

745         "Wait_min",
746         "Detour_legs",
747         "Latest_return",
748     ]
749 )
750 for case_name, scenarios in results.items():
751     for scenario_name, item in scenarios.items():
752         writer.writerow(
753             [
754                 case_name,
755                 scenario_name,
756                 item["N"],
757                 item["stage_a"]["Tmax_star_h"],
758                 item["stage_b"]["epsilon"],
759                 item["Tmax"],
760                 item["Tmin"],
761                 item["delta"],
762                 item["total_wait_min"],
763                 sum(route["detour_leg_count"] for route in
764                     ↪ item["routes"]),
765                 item["latest_return"],
766             ]
767         )
768
769 def main() -> None:
770     parser = argparse.ArgumentParser()
771     parser.add_argument("--points", type=Path, required=True)
772     parser.add_argument("--zones", type=Path, required=True)
773     parser.add_argument("--q2-solution", type=Path, required=True)
774     parser.add_argument(
775         "--baseline-q3-solution",
776         type=Path,
777         help="可选: 算法A的 q3_solution.json;
778             ↪ 提供后作为算法B的必选初始种子",
779     )
780     parser.add_argument("--output-dir", type=Path, required=True)
781     parser.add_argument("--epsilon", type=float, default=0.005)
782     parser.add_argument("--ring-samples", type=int, default=16)
783     parser.add_argument("--safety-margin", type=float, default=0.0)
784     parser.add_argument("--stage-a-iterations", type=int,
785         ↪ default=800)
786     parser.add_argument("--stage-b-iterations", type=int,
787         ↪ default=800)
788     parser.add_argument("--seconds-per-case", type=float,
789         ↪ default=45.0)
790     parser.add_argument("--extra-uavs", type=int, default=0)
791     args = parser.parse_args()
792     if not 0.0 <= args.epsilon <= 0.05:
793         raise ValueError("epsilon 建议位于 [0,0.05]")
794     if args.extra_uavs < 0:
795         raise ValueError("extra-uavs 不能为负")
796
797     cases = baseline.load_cases(args.points, args.zones)
798     q2 = json.loads(args.q2_solution.read_text(encoding="utf-8"))
799     assert set(cases) == set(q2)
800     baseline_q3 = None
801     if args.baseline_q3_solution is not None:
802         baseline_q3 = json.loads(args.baseline_q3_solution.read_text(
803             ↪ encoding="utf-8"))
804     assert set(cases) == set(baseline_q3)
805     results: dict[str, dict] = {}
806     for case_index, (case_name, data) in enumerate(cases.items()):
807         seed_source = baseline_q3[case_name] if baseline_q3 is not
808             ↪ None else q2[case_name]
809         physical = [
810             [int(value) for value in route["sequence"][1:-1]]
811             for route in seed_source["routes"]
812         ]
813         initial, mapping = build_unique_copies(physical)
814         expected = expected_copy_ids(data)
815         if set(mapping) != expected:
816             raise RuntimeError(f"{case_name}:
817                 ↪ 问题二路线与任务副本需求不一致")
818     scenarios: dict[str, dict] = {}
819     scenario_routes = [route[:] for route in initial]
820     for extra in range(args.extra_uavs + 1):
821         scenario_name = "formal_fixed_NO" if extra == 0 else
822             ↪ f"sensitivity_NO_plus_{extra}"
823         result = solve_case_n(
824             case_name,
825             data,
826             scenario_routes,
827             mapping,
828             args.epsilon,
829             args.ring_samples,
830             args.safety_margin,
831             args.stage_a_iterations,
832             args.stage_b_iterations,
833             args.seconds_per_case,
834             20260818 + 1000 * case_index + extra,
835         )
836     scenarios[scenario_name] = result
837     print(
838         f"{case_name} {scenario_name}: N={result['N']} "
839         f"Tmax*={result['stage_a']['Tmax_star_h']:.6f} "
840         f"Tmax={result['Tmax']:.6f} "
841         ↪ delta={result['delta']:.6f} "
842         f"wait={result['total_wait_min']:.3f}",
843         flush=True,
844     )
845     # 机队规模敏感性采用逐级热启动:
846     ↪ 下一规模从当前已优化方案拆出一条
847     # 新路线, 而不是每次回到未经优化的原始路线重新硬折。
848     if extra < args.extra_uavs:
849         optimized_routes = [route["task_sequence"][1:-1] for
850             ↪ route in result["routes"]]
851         scenario_routes = add_uavs(optimized_routes, 1)
852     results[case_name] = scenarios
853
854     args.output_dir.mkdir(parents=True, exist_ok=True)
855     output = args.output_dir / "q3_enhanced_solution.json"
856     output.write_text(json.dumps(results, ensure_ascii=False,
857         ↪ indent=2), encoding="utf-8")
858     write_summary(results, args.output_dir)
859     print(output)
860
861 if __name__ == "__main__":
862     main()

```

问题三快速独立校验程序

路径: submission_package/question3/program/q3_fast_validator.py

```

1  """ 问题三保存结果的独立快速复核。
2
3  先用线包包围盒与圆包围盒作常数时间筛查, 仅对可能相交的航段求解析交区间;
4  再逐事件校验飞行、服务和等待均不与正时长管制区冲突。
5  """
6
7  from __future__ import annotations
8
9  import argparse
10 import json
11 import math
12 from collections import Counter
13 from datetime import datetime
14 from pathlib import Path
15
16 import pandas as pd
17
18 REQ = {"I": 3, "II": 2, "III": 1}
19 TOL = 1.0e-7
20
21 def to_min(text: object) -> float:
22     value = datetime.strptime(str(text).strip(), "%H:%M")
23     return (value.hour - 8) * 60 + value.minute
24
25 def overlap(a0: float, a1: float, b0: float, b1: float) -> bool:
26     return max(a0, b0) < min(a1, b1) - TOL

```



```

32 def bbox_maybe(a, b, center, radius) -> bool:
33     return not (
34         max(a[0], b[0]) < center[0] - radius
35         or min(a[0], b[0]) > center[0] + radius
36         or max(a[1], b[1]) < center[1] - radius
37         or min(a[1], b[1]) > center[1] + radius
38     )
39
40
41 def segment_inside_interval(a, b, center, radius):
42     if not bbox_maybe(a, b, center, radius):
43         return None
44     dx, dy = b[0] - a[0], b[1] - a[1]
45     fx, fy = a[0] - center[0], a[1] - center[1]
46     aa = dx * dx + dy * dy
47     bb = 2 * (fx * dx + fy * dy)
48     cc = fx * fx + fy * fy - radius * radius
49     disc = bb * bb - 4 * aa * cc
50     if disc < -TOL:
51         return None
52     disc = max(0.0, disc)
53     lo = max(0.0, (-bb - math.sqrt(disc)) / (2 * aa))
54     hi = min(1.0, (-bb + math.sqrt(disc)) / (2 * aa))
55     return None if lo > hi + TOL else (lo, hi)
56
57
58 def main() -> None:
59     parser = argparse.ArgumentParser()
60     parser.add_argument("--points", type=Path, required=True)
61     parser.add_argument("--zones", type=Path, required=True)
62     parser.add_argument("--solution", type=Path, required=True)
63     args = parser.parse_args()
64
65     result = json.loads(args.solution.read_text(encoding="utf-8"))
66     points_book = pd.ExcelFile(args.points)
67     zones_book = pd.ExcelFile(args.zones)
68     assert set(result) == set(points_book.sheet_names) ==
        ↳ set(zones_book.sheet_names)
69
70     for case_name in points_book.sheet_names:
71         points_df = pd.read_excel(points_book, sheet_name=case_name)
72         zones_df = pd.read_excel(zones_book, sheet_name=case_name)
73         coords = {0: (0.0, 0.0)}
74         expected = {}
75         for _, row in points_df.iterrows():
76             point_id = int(row.Point_ID)
77             coords[point_id] = (float(row.X_Coordinate),
        ↳ float(row.Y_Coordinate))
78             expected[point_id] = REQ[str(row.Inspection_Level)]
79         zones = []
80         for _, row in zones_df.iterrows():
81             start, end = to_min(row.Start_Time), to_min(row.End_Time)
82             if end > start + TOL:
83                 zones.append((str(row.Zone_ID), (float(row.Center_X),
        ↳ float(row.Center_Y)), float(row.Radius), start,
        ↳ end))
84
85         found = Counter()
86         times = []
87         for route in result[case_name]["routes"]:
88             seq = [int(x) for x in route["sequence"]]
89             assert seq[0] == seq[-1] == 0
90             assert all(a != b for a, b in zip(seq[1:-1], seq[2:-1]))
91             found.update(seq[1:-1])
92             last_end = 0.0
93             for event in route["events"]:
94                 start, end = float(event["start_min"]),
        ↳ float(event["end_min"])
95                 assert abs(start - last_end) < 2.0e-6
96                 assert end >= start - TOL
97                 if event["type"] == "flight":
98                     a, b = coords[int(event["from"])],
        ↳ coords[int(event["to"])]
99                     for zone_id, center, radius, z0, z1 in zones:
100                         interval = segment_inside_interval(a, b,
        ↳ center, radius)
101                         if interval is None:
102                             continue
103                         lo, hi = interval
104
105                         assert not overlap(start + lo * (end -
        ↳ start), start + hi * (end - start), z0,
        ↳ z1), f"{case_name} UAV{route['uav']}"
106                         ↳ flight conflicts {zone_id}"
107                         elif event["type"] in {"service", "wait"}:
108                             node = int(event["node"])
109                             if event["type"] == "wait" and node == 0:
110                                 last_end = end
111                                 continue
112                             point = coords[node]
113                             for zone_id, center, radius, z0, z1 in zones:
114                                 if math.dist(point, center) <= radius + TOL:
115                                     assert not overlap(start, end, z0, z1),
        ↳ f"{case_name} UAV{route['uav']}"
116                                     ↳ {event['type']} conflicts {zone_id}"
117                             last_end = end
118                             assert abs(last_end / 60 - float(route["time_h"])) <
        ↳ 2.0e-6
119                             times.append(last_end / 60)
120                             assert dict(found) == expected
121                             case = result[case_name]
122                             assert int(case["N"]) == len(case["routes"])
123                             assert abs(max(times) - float(case["Tmax"])) < 2.0e-6
124                             assert abs(min(times) - float(case["Tmin"])) < 2.0e-6
125                             assert abs(max(times) - min(times) - float(case["delta"])) <
        ↳ 2.0e-6
126                             print(f"{case_name}: PASSED")
127
128 if __name__ == "__main__":
129     main()

```

问题三算法 B 逐事件独立验证程序

路径: submission_package/question3/program/q3_enhanced_validator.py

```

1 """ 问题三增强算法的独立逐事件验证器。"""
2
3 from __future__ import annotations
4
5 import argparse
6 import json
7 from collections import Counter
8 from pathlib import Path
9
10 import q3_solver as baseline
11 from q3_solver_enhanced import expected_copy_ids, task_point
12
13
14 TOL = 2.0e-6
15
16
17 def main() -> None:
18     parser = argparse.ArgumentParser()
19     parser.add_argument("--points", type=Path, required=True)
20     parser.add_argument("--zones", type=Path, required=True)
21     parser.add_argument("--solution", type=Path, required=True)
22     args = parser.parse_args()
23
24     cases = baseline.load_cases(args.points, args.zones)
25     result = json.loads(args.solution.read_text(encoding="utf-8"))
26     assert set(result) == set(cases)
27     for case_name, scenarios in result.items():
28         data = cases[case_name]
29         expected = expected_copy_ids(data)
30         for scenario_name, case in scenarios.items():
31             found_tasks: list[str] = []
32             times: list[float] = []
33             total_wait = 0.0
34             for route in case["routes"]:
35                 task_sequence = route["task_sequence"]
36                 assert task_sequence[0] == task_sequence[-1] ==
        ↳ "BASE"
37                 tasks = task_sequence[1:-1]
38                 found_tasks.extend(tasks)

```

```

39     physical = [task_point(task) for task in tasks]      103     main()
40     assert route["sequence"] == [0, *physical, 0]
41     assert all(a != b for a, b in zip(physical,
    ↪ physical[1:]))
42
43     last_end = 0.0
44     services: list[str] = []
45     distance_km = 0.0
46     for event in route["events"]:
47         start = float(event["start_min"])
48         end = float(event["end_min"])
49         assert abs(start - last_end) < TOL
50         assert end >= start - TOL
51         if event["type"] == "flight":
52             a = tuple(float(x) for x in event["from_xy"])
53             b = tuple(float(x) for x in event["to_xy"])
54             distance_km += float(event["distance_km"])
55             for zone in data.zones:
56                 if not zone.active:
57                     continue
58                 interval =
    ↪ baseline.segment_circle_interval(a,
    ↪ b, zone)
59                 if interval is None:
60                     continue
61                 lo, hi = interval
62                 assert not baseline.overlaps(
63                     start + lo * (end - start),
64                     start + hi * (end - start),
65                     zone.start_min,
66                     zone.end_min,
67                     ), f"{case_name}/{scenario_name}/UAV{rou
    ↪ te['uav']} flight conflicts
    ↪ {zone.zone_id}"
68             elif event["type"] in {"wait", "service"}:
69                 xy = tuple(float(x) for x in
    ↪ event["node_xy"])
70                 if event["type"] == "wait":
71                     total_wait += end - start
72                 else:
73                     services.append(event["task_id"])
74                     assert abs((end - start) -
    ↪ baseline.SERVICE_MIN) < TOL
75                 for zone in data.zones:
76                     if zone.active and
    ↪ baseline.point_inside(xy, zone):
77                         assert not baseline.overlaps(start,
    ↪ end, zone.start_min,
    ↪ zone.end_min), (
78                             f"{case_name}/{scenario_name}/UA
    ↪ V{route['uav']} "
79                             f"{event['type']} conflicts
    ↪ {zone.zone_id}"
80                     )
81                 else:
82                     raise AssertionError(f"未知事件类型
    ↪ {event['type']}")
83                 last_end = end
84                 assert services == tasks
85                 assert abs(distance_km - float(route["distance_km"]))
    ↪ < TOL
86                 assert abs(last_end / 60.0 - float(route["time_h"]))
    ↪ < TOL
87                 times.append(float(route["time_h"]))
88
89     assert len(found_tasks) == len(set(found_tasks))
90     assert set(found_tasks) == expected
91     assert int(case["N"]) == len(case["routes"])
92     assert abs(max(times) - float(case["Tmax"])) < TOL
93     assert abs(min(times) - float(case["Tmin"])) < TOL
94     assert abs(max(times) - min(times) -
    ↪ float(case["delta"])) < TOL
95     assert abs(total_wait - float(case["total_wait_min"])) <
    ↪ TOL
96     limit = float(case["stage_b"]["Tmax_limit_h"])
97     assert float(case["Tmax"]) <= limit + TOL
98     assert case["deadline_17_00"] is False
99     print(f"{case_name}/{scenario_name}: PASSED")
100
101
102 if __name__ == "__main__":

```